

SAPIENZA
UNIVERSITÀ DI ROMA

A Vision-Guided Reinforcement Learning Approach for Pick-and-Place Task Execution on the TIAGo Pro Platform

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Artificial Intelligence and Robotics (AIRO)

Lorenzo Barbieri
ID number 2144036

Advisor
Prof. Daniele Nardi

Co-Advisor
Vincenzo Suriani

Academic Year 2025/2026

A Vision-Guided Reinforcement Learning Approach for Pick-and-Place Task Execution on the TIAGo Pro Platform
Master Thesis. Sapienza University of Rome

© 2026 Lorenzo Barbieri. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: barbieri.2144036@studenti.uniroma1.it, lorenzbarbieri2408@gmail.com

Contents

1 Introduction **1**
1.1 Problem Context 1
1.2 Task Description and Motivations 2
1.3 Technical Challenges 3
1.4 Proposed Solution 4
1.5 Structure of the Thesis 5

2 Background and Related Work **6**
2.1 Background 6
2.1.1 Reinforcement Learning: Foundations 6
2.1.2 Proximal Policy Optimization (PPO) 10
2.1.3 PAL Robotics and the TIAGo Pro Platform 11
2.2 Related Work 13
2.2.1 Learning-Based Approaches to Robotic Manipulation 13
2.2.2 Reinforcement Learning for Robotic Manipulation: Overview 14
2.2.3 Sim-to-Real Transfer 15
2.2.4 Vision-Based Manipulation with RL 16
2.2.5 RL Works on TIAGo Family Platforms 17

3 Methodology **19**
3.1 Reinforcement Learning Framework 19
3.1.1 The mjlab Framework 19
3.1.2 Simulation Environment 20
3.1.3 PPO Implementation 21
3.1.4 Observation and Action Spaces 22
3.1.5 Reward Design 23
3.1.6 Success and Termination Conditions 26
3.1.7 Bridging the Sim-to-Real Gap 27
3.1.8 Training Summary 29
3.2 Visual Perception Pipeline 30
3.2.1 Pipeline Overview 30
3.2.2 YOLO Object Detection 31
3.2.3 Colour Segmentation of the Region of Interest 34
3.2.4 Depth Filtering and Back-projection 35
3.2.5 Pose and Yaw Estimation 36
3.3 ROS 2 Deployment Architecture 37

3.3.1 Nodes Architecture 38
3.3.2 Vision Node 39
3.3.3 Grasp Detection and Kinematic Override 39
3.3.4 Control Nodes 40
3.3.5 Execution Frequencies 41

4 Experimental Results **42**
4.1 Policy Training Metrics and Convergence Analysis 42
4.1.1 Task Reward Density Progression 42
4.1.2 Total Reward Density and PPO Loss Analysis 44
4.2 Policy Performance Evaluation in mjlab 44
4.2.1 Robustness and Generalization Analysis 45
4.2.2 Evaluation Conclusions 46
4.3 Evaluation Through Gazebo Implementation 46
4.3.1 3D Pose Estimation Accuracy vs. Ground-Truth 47
4.4 Evaluation on the TIAGo Pro Platform 49
4.4.1 Experimental Setup 49
4.4.2 Qualitative Evaluation and Operational Observations 50
4.4.3 Sim-to-Real Discrepancies and Operational Limitations . . . 50
4.4.4 Quantitative Execution Analysis 51

5 Conclusions and Future Work **55**
5.1 Conclusions 55
5.2 Limitations 55
5.3 Future Work 56
5.3.1 Transfer to the Kangaroo Platform 56
5.3.2 Further Work on the TIAGo Pro 57

List of Figures

1.1 PAL Robotics platforms relevant to this work 2

2.1 Agent–environment interaction loop in an MDP 7
2.2 PAL Robotics TIAGo Pro mobile manipulator platform 12
2.3 Kinematic structure of the TIAGo Pro arm 13
2.4 TIAGo and TIAGo++ platforms 18

3.1 Simulated training environment in mjlab 20
3.2 Actor and critic architectures 22
3.3 Dataset samples 31
3.4 YOLO detections 33
3.5 Detection during deployment 34
3.6 HSV segmentation 35
3.7 Depth filtering 36
3.8 Oriented bounding box fit 37
3.9 Deployment architecture 38
3.10 Grasp detection state machine 40

4.1 Task reward density progression during training 43
4.2 PPO training metrics in mjlab 44
4.3 Fingertip yaw alignment error schematic 45
4.4 TIAGo Pro model in the Gazebo environment 47
4.5 RViz2 visualization during Gazebo evaluation 48
4.6 Real-robot evaluation environment and test objects 49
4.7 Sequential keyframes of real-world task execution 50
4.8 End-effector and object centroid trajectories during deployment . . . 52
4.9 Right-arm joint positions during physical deployment 53
4.10 Right-arm joint efforts during physical deployment 54

List of Tables

3.1 Reward terms 26
3.2 PPO hyperparameters 30
3.3 YOLO validation metrics 33
3.4 ROS 2 topic rates 41

4.1 Policy evaluation results in mjlab across 1,000 trials 46
4.2 Gazebo pose prediction accuracy metrics 48

Abstract

This thesis presents a reinforcement learning framework for pick-and-place on the TIAGo Pro mobile manipulator, covering the complete workflow from training in simulation to deployment on the real robot. The task consists of reaching an object lying on a table, grasping it, carrying it to a chest and releasing it inside, and then returning for the next object, so that the cycle can be repeated without interruption. The robot relies solely on its onboard hardware, namely the head-mounted RGB-D camera and the proprioceptive sensors. This raises an occlusion problem: as the arm and the gripper approach and grasp the object, they hide it from the camera, which makes policies that act directly on images unreliable.

The proposed method therefore keeps perception and control separate. A perception pipeline, which combines a learned object detector with classical image processing, estimates the 3D position and the yaw of the object. A reinforcement learning policy, trained with Proximal Policy Optimization in the **mjlab** simulator, receives this estimate together with the state of the robot and commands the arm and the gripper. Once a grasp is detected from the motor current of the gripper, the estimated object pose is replaced by the pose of the end-effector, so that the task no longer depends on the camera.

The method was validated in successive steps: the policy in simulation under observation noise, the perception pipeline in Gazebo, and the complete system through a ROS 2 architecture, first in Gazebo and then on the real robot. There the policy, trained only in simulation, was deployed without any fine-tuning and handled a variety of objects.

Chapter 1

Introduction

1.1 Problem Context

Robotic manipulation represents one of the foundational capabilities in modern robotics, enabling autonomous agents to physically interact with objects in their environment. Among manipulation skills, **Pick-and-Place** serves as the canonical, fundamental building block. A pick-and-place task requires a robot to reach a target object, establish a stable grasp using an end-effector, transport it through free space, and release it at a designated goal location.

Performing this task robustly with a multi-joint robotic arm is a challenging problem: many joints have to be coordinated at once, the contact between the gripper and the object is difficult to predict, and objects rarely appear twice in the same configuration. Localizing the target object in 3D space is itself a non-trivial requirement, making visual perception, typically through RGB-D cameras, a central component of the manipulation pipeline. Traditional approaches, in which the motion of the arm is planned and controlled on the basis of explicit models, perform well in rigidly structured environments where object positions are known in advance. However, they rely on the designer's ability to specify accurate models of the robot, the objects, and their interactions ahead of time, an assumption that breaks down as soon as objects vary in location, shape, or appearance, or when contact with the environment cannot be fully predicted or modeled [12].

These limitations have motivated the robotics community to explore **learning-based approaches** that acquire manipulation skills directly from data or experience, reducing reliance on explicit analytical models of the system or environment. Kroemer et al. [12] frame this shift around a central premise: the real world contains too much variation for a robot to have an accurate a priori model of its environment, the objects in it, or the skills required to manipulate them.

In a learning-based approach, the behavior of the robot is not programmed explicitly. The designer provides the means to learn it, for instance examples of the task being performed or a measure of how well it is being carried out, and an algorithm progressively improves the behavior on that basis. The result is a *policy*: the rule by which the robot chooses its next action from what it currently observes. Since the policy is queried many times per second, the robot reacts continuously to the situation in front of it instead of executing a motion planned in advance.

The control strategy adopted in this thesis is *Reinforcement Learning* (RL). In RL the robot is given no examples of the task: it learns by trial and error, guided only by a *reward*, a number that scores how well the task is being carried out. Actions that lead to a higher reward become more likely, and over many attempts the policy converges to a behavior that accomplishes the task. The drawback of RL is the amount of practice it requires, far more than can reasonably be collected on a physical robot. For this reason the policy is trained in simulation, where many copies of the robot can practice in parallel. A simulator, however, is only an approximation of reality, and a policy that succeeds in simulation is not guaranteed to succeed on the real robot. This discrepancy, known as the *sim-to-real gap*, shapes many of the design choices presented in this work.

1.2 Task Description and Motivations

This work was carried out during my internship at **PAL Robotics**. At that time, the company's exploration of control techniques based on Deep Reinforcement Learning (Deep RL) was directed mainly at the locomotion of humanoid robots, whereas manipulation was still carried out almost exclusively with classical control techniques. On the **Kangaroo** humanoid, this approach had already given encouraging results in a wide variety of locomotion tasks, including walking, jumping and dancing. Following these successes the main goal of the internship was therefore to extend RL-based control to manipulation, by designing a Deep RL framework for the **TIAGo Pro** mobile manipulator. The investigated approach would then serve as a reference for future analogous work on the Kangaroo platform. Both platforms are shown in Figure 1.1.



(a) TIAGo Pro mobile manipulator **(b)** Kangaroo humanoid robot

Figure 1.1. The two PAL Robotics platforms relevant to this work. The TIAGo Pro (left) served as the development and deployment testbed, while the Kangaroo humanoid (right) is the ultimate target platform for which this preliminary manipulation pipeline lays the groundwork.

The core task assigned was the development of an RL policy capable of reaching a target object placed on a table, grasping it, and transporting it to a desired goal position. The requested policy had to be trained within **mjlab** [34], a recent open-source framework for robot learning that simulates a large number of environments in parallel on the GPU (Section 3.1.1). Completing this task required, in addition to the policy itself, a **visual perception component** to estimate the object's state in 3D space, namely its position and orientation, from camera data.

The project was structured into two sequential development stages. The first focused on establishing a baseline pipeline capable of reaching and grasping a cuboid object placed at a random position on a table and transporting it to a variable goal location. Building on this initial milestone, the task was extended into a complete pick-and-place system, requiring the robot to transport and release the grasped object inside a storage chest, which serves as the core pipeline analyzed in this thesis.

1.3 Technical Challenges

Carrying out this task raised multiple technical challenges which shaped the solution presented in Section 1.4.

The first is sensing. The work relies only on the hardware the robot is delivered with, so the head-mounted camera is the only source of information on the object: given the distance between the camera and the table, the perception pipeline had to extract crucial informations on objects that occupy just a small portion of the image. Additionally, the sight of view of the object gets severely occluded in the most delicate phases of the task: the arm and the gripper during the final stages of the approach and during the picking operations almost completely obscure the object from the camera images. The gripper, moreover, has no force or tactile sensor, so the robot has no direct way of knowing whether it is holding the object.

The second is learning the task. Pick-and-place is long and made of distinct phases, reaching, grasping, carrying and releasing, which a single policy has to learn from a reward designed by hand. The main difficulty is that a phase may require a behavior that contradicts what the policy has learned in the previous ones: the gripper must stay open during the approach and close on the object for the grasp, and after learning that a firm hold is what is rewarded throughout the transport, the policy has to learn to let the object go at the goal. The reward therefore has to make each behavior appear at the right moment without undoing the others.

The third is the sim-to-real gap. The policy is trained entirely in simulation and has to work on the real robot without further training, and nothing guarantees that it will: a policy that performs flawlessly in simulation can fail outright once it is deployed. The reason is that the policy learns whatever works in the simulator, including behaviors that depend on its approximations. Contacts, friction, the response of the motors and the delays of the control loop are all modeled only approximately, and the policy has no way of knowing which of the regularities it relies on will still hold on the real robot. The gap is wider still when the task depends on visual perception: simulated images differ from real ones in lighting, textures, reflections and sensor noise, so that what is learned from the simulated scene may

no longer hold in front of the real one.

The fourth is integration: perception and control run at different rates, and have to be combined into a single loop that operates in real time on the robot.

1.4 Proposed Solution

In order to address the challenges described in Section 1.3, the proposed approach is organized into the following four components:

1. **RL Policy:** The policy is trained entirely in simulation, within **mjlab**, using the Proximal Policy Optimization (PPO) algorithm [27]. Given the state of the robot, the estimated pose of the object and the goal position, it commands the joints of the arm and the gripper. To prepare it for the real robot, the physical properties of the simulated scene are varied during training and noise is added to what the policy observes, so that it can be transferred to the real robot without any further training.

2. **Perception Pipeline:** The pose of the object is estimated from the color and depth images of the head-mounted camera. A YOLO object detector first locates the object in the image; a sequence of classical image processing steps then isolates its pixels and uses their depth to recover its 3D position and its orientation.

3. **Grasp Detection:** Once the gripper closes around the object, the camera can no longer provide a reliable estimate of its pose. The system therefore detects the grasp from the motor current of the gripper and, from that moment until the object is released, replaces the visual estimate with the pose of the end-effector, which the robot knows from its own joint readings.

4. **ROS 2 Deployment:** The components above are connected through ROS 2, the software framework on which the TIAGo Pro operates, into a single closed loop that goes from the camera to the joint controllers of the robot and runs in real time. The loop was tested first in the Gazebo simulator and then on the real robot.

The proposed method is therefore characterized by a separation between **perception and control**: the policy never receives the camera images, only the pose of the object estimated from them. This separation yields three advantages:

- **Robustness to Occlusion:** A policy that received the camera images directly would be left without useful information while the arm hides the object. The perception module instead provides an estimate whenever the object is visible and holds the last one while it is hidden. If a grasp attempt fails or the object is displaced, the estimate is updated as soon as the object comes back into view, and the policy adjusts its motion accordingly.

- **Independence from Visual Appearance:** Since the policy never sees an image, it is not affected by the differences in rendering, lighting and camera noise between simulation and reality.

- **Faster Training:** No image has to be rendered or processed while the policy is being trained, which greatly reduces the computational cost of the simulation.

1.5 Structure of the Thesis

The remainder of this thesis document is organized into the following chapters:

- **Chapter 2: Background and Related Work** introduces the foundations of reinforcement learning and the PPO algorithm, presents the TIAGo Pro platform, and reviews the works related to this thesis, starting from a comparison of the main learning-based approaches to manipulation.
- **Chapter 3: Methodology** describes the three components of the proposed system: the reinforcement learning framework, covering the training environment in **mjlab**, the PPO implementation, the observation and action spaces, the reward design, the termination conditions and the domain randomization; the perception pipeline, covering YOLO 2D detection, colour segmentation of the region of interest, depth filtering, back-projection and pose estimation; and the ROS 2 deployment architecture, covering the role of each node and the detection of the grasp.
- **Chapter 4: Experimental Results** analyses the training curves of the policy and evaluates its performance in **mjlab**, the accuracy of the pose estimation in Gazebo, and the deployment on the real TIAGo Pro.
- **Chapter 5: Conclusions and Future Work** summarizes the work, discusses the limitations that emerged and outlines future developments, both on the TIAGo Pro and towards implementations on the Kangaroo humanoid robot.

Chapter 2

Background and Related Work

This chapter provides a theoretical background and literature synthesis to ground the vision-guided reinforcement learning (RL) manipulation framework developed in this thesis. Section 2.1 presents the foundational concepts of Reinforcement Learning followed by a detailed mathematical formulation of Proximal Policy Optimization (PPO) and an overview of PAL Robotics and the TIAGo Pro mobile manipulator platform. Section 2.2 then compares the main learning-based approaches to robotic manipulation, traces how reinforcement learning came to be applied to it, reviews how the gap between simulation and hardware is attacked, how perception is brought into the control loop, and what has been done on the TIAGo family of platforms.

2.1 Background

2.1.1 Reinforcement Learning: Foundations

Learning from Interaction

In Reinforcement Learning (RL) an agent learns to act by interacting with an environment: it observes a state, chooses an action, and receives a scalar reward that evaluates what it did. No one tells it which action would have been correct, as a label would in supervised learning; the reward only scores the action taken, leaving the agent to discover the rest by trial and error. Because a single decision alters future states, the agent must optimize for delayed, cumulative rewards rather than immediate gains. This dynamic introduces the central challenge of RL: balancing the exploitation of known strategies with the exploration of untested actions that might yield better long-term outcomes.

Structurally, an RL framework relies on four core components beyond the agent and environment [29]. The *policy* dictates behavior by mapping perceived states to specific actions. The *reward signal* provides immediate evaluative feedback. The *value function* estimates the total cumulative reward expected from a given state; this ensures the agent can prioritize future payoffs over immediate, short-sighted gains. Finally, an optional *model* predicts environment dynamics, such as state transitions, to enable proactive planning. Algorithms that bypass this predictive step to learn strictly through direct interaction are classified as *model-free*, the category to which the methodology of this thesis belongs.

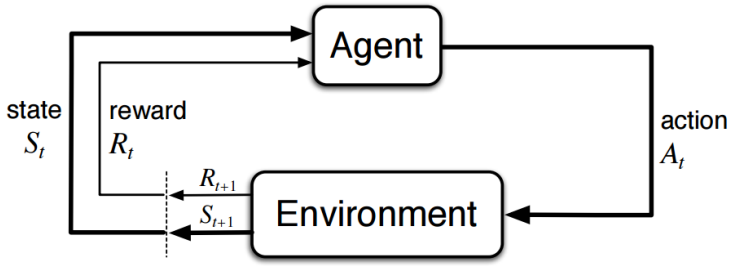


Figure 2.1. Agent–environment interaction loop in an MDP (from [29, Fig. 3.1]).

Markov Decision Processes

The interaction between agent and environment is formalized as a *Markov Decision Process* (MDP). Unless otherwise stated, the definitions in this section follow Sutton and Barto [29]. An MDP is defined by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \tag{2.1}$$

where:

- $\mathcal{S}$ is the *state space*, the set of all possible configurations of the environment;
- $\mathcal{A}$ is the *action space*, the set of all control inputs available to the agent;
- $P(s' \mid s, a)$ is the *transition function*, the probability of reaching state s' after taking action a in state s;
- $R(s, a, s')$ is the *reward function*, the scalar feedback associated with that transition;
- $\gamma \in [0, 1)$ is the *discount factor*, which sets how much future rewards count relative to immediate ones.

The interaction unfolds in discrete time steps (Fig. 2.1). At step t the agent observes the state s_t and selects an action a_t; the environment responds by moving to a new state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$ and emitting the reward $r_t = R(s_t, a_t, s_{t+1})$.

The reward is the only channel through which the task is communicated to the agent.

The *Markov property* requires the next state to depend only on the current state and action, and not on the earlier history:

$$\Pr\{s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0\} = P(s_{t+1} \mid s_t, a_t). \tag{2.2}$$

It is best read as a requirement on the state: s_t must summarize everything about the past that is relevant to the future.

On a real robot such a state is rarely accessible. Joint encoders, a camera and a pose estimate describe the scene only partially, so what the agent observes is a noisy and incomplete function of it. Problems of this kind are modeled as a *Partially Observable Markov Decision Process* (POMDP) [10], in which the agent receives at each step an observation $o_t \sim O(\cdot \mid s_t)$ carrying incomplete or corrupted information about s_t.

Policy and the Optimization Objective

The agent's behavior is formalized by a *policy* $\pi(a \mid s)$, which specifies the probability density of selecting action a given state s. In continuous action spaces, π is typically modeled as a parameterized Gaussian distribution whose mean and variance are predicted conditioned on the current state.

The interaction generates a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$. Rather than optimizing for instantaneous feedback, the agent aims to maximize the *return* G_t, defined as the discounted sum of future rewards accumulated from step t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k} = r_t + \gamma \, G_{t+1}. \tag{2.3}$$

The discount factor $\gamma \in [0, 1)$ ensures convergence over infinite horizons and reflects a preference for immediate over distal rewards. The formal objective of Reinforcement Learning is therefore to identify an optimal policy π^* that maximizes the expected return of the whole trajectory, i.e. the return G_0 accumulated from the initial step $t = 0$:

$$\pi^* = \arg\max_{\pi} J(\pi), \qquad J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[G_0 \right]. \tag{2.4}$$

Value Functions and the Advantage

Evaluating states and actions without unrolling entire trajectories relies on *value functions*. The *state-value function* $V^\pi(s)$ represents the expected return when starting from state s and following policy π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid s_t = s \right]. \tag{2.5}$$

Similarly, the *action-value function* $Q^\pi(s, a)$ quantifies the expected return of taking an arbitrary action a in state s and following π thereafter:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid s_t = s, \, a_t = a \right]. \tag{2.6}$$

Substituting the recursive form of the return, $G_t = r_t + \gamma \, G_{t+1}$, into these definitions gives the fundamental *Bellman expectation equation*, which relates the value of the current state to the expected values of its successors:

$$V^\pi(s) = \mathbb{E}_\pi \left[r_t + \gamma \, V^\pi(s_{t+1}) \mid s_t = s \right]. \tag{2.7}$$

This relation is what makes *bootstrapping* possible, that is estimating a value partly from another estimate, instead of only from rewards actually observed. Rather than waiting for the episode to end and summing all the rewards, the agent takes the reward of the next step and adds its own current estimate of the value of the state it lands in.

While $V^\pi(s)$ evaluates the baseline quality of a state, policy optimization requires isolating the relative merit of a specific decision. This is captured by the *advantage function* [26]:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \tag{2.8}$$

The advantage measures how much better action a is compared to the expected performance of π in state s. Beyond serving as an intuitive metric, subtracting the state baseline $V^{\pi}(s)$ is crucial during optimization: it significantly reduces the variance of policy gradient estimates without introducing bias.

Because continuous robotics domains render tabular representations intractable, both policies and value functions are approximated using deep neural networks [13].

Taxonomy of Model-Free Methods

Model-free algorithms differ in what they learn and in how an action is obtained from it. Three families can be distinguished, although the boundaries between them are not sharp:

- **Value-based methods** learn an approximation of the optimal action-value function $Q^*(s, a)$ by bootstrapping on the Bellman optimality equation, as in Q-learning and its deep variant DQN [17]. The policy is implicit: the agent acts greedily, $a_t = \arg\max_a Q(s_t, a)$. With a finite action set this maximization is a simple enumeration. With continuous actions it has no closed form for a neural Q and must be approximated at every control step, for example with a sampling-based optimizer as in QT-Opt [11]. Discretizing the action space instead scales exponentially with its dimension: ten bins per joint on a 7-DoF arm already yield 10^7 discrete actions.

- **Policy-gradient methods** parameterize the policy directly as $\pi_\theta(a \mid s)$ and update θ by gradient ascent on $J(\pi_\theta)$, using the policy gradient theorem [30]. For continuous actions the network outputs the parameters of a distribution, typically the mean of a Gaussian with a diagonal covariance, and an action is obtained by sampling from it, so no maximization over actions is needed. In their basic form, such as REINFORCE [33], the gradient is estimated from Monte Carlo returns of complete episodes. This estimate is unbiased but has high variance, which makes learning slow and sample-hungry.

- **Actor–critic methods** are policy-gradient methods that additionally learn a value function (the *critic*) and use it in place of the Monte Carlo return to train the policy (the *actor*). Bootstrapping from the critic lowers the variance of the gradient estimate at the cost of some bias. The two common designs differ in what the critic estimates. In PPO [27] the critic is a state-value function $V(s)$, used as the baseline of the advantage (Section 2.1.2). In Soft Actor-Critic (SAC) [4] the critic is an action-value function $Q(s, a)$, and the actor is trained to output actions that maximize it.

Algorithms further divide into *on-policy* and *off-policy*, according to whether the data used for an update comes from the policy being updated. Off-policy algorithms, such as DQN, DDPG, TD3 and SAC, optimize the current policy using transitions sampled from historical data stored in a replay buffer. While this reuse yields high sample efficiency, learning from an out-of-date distribution complicates stability and increases hyperparameter sensitivity. Conversely, on-policy algorithms, such as TRPO and PPO, evaluate and improve solely the policy that gathered the data, discarding transitions immediately after the update. Although traditionally

considered data inefficient, on-policy updates provide superior numerical stability. In modern massively parallel simulations, where generating fresh trajectories across thousands of environment instances is computationally inexpensive, this sample-efficiency trade-off becomes favorable.

These architectural and algorithmic considerations motivate the choice of Proximal Policy Optimization (PPO), an on-policy actor–critic algorithm detailed in the next section.

2.1.2 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [27] is an **on-policy actor-critic** algorithm that has become the dominant choice for continuous robotic control due to its empirical stability, simple implementation, and compatibility with massively parallel GPU simulation. PPO maintains two neural networks trained jointly:

- An **Actor** network $\pi_\theta(a \mid s)$, parameterized by θ, which represents the stochastic policy and outputs a Gaussian distribution over continuous actions given the current state. Sampling from this distribution provides exploration by construction, and its standard deviation is learned along with the mean.
- A **Critic** network $V_\phi(s)$, parameterized by ϕ, which estimates the expected discounted return from a given state and is used to compute low-variance advantage estimates for the Actor's gradient updates.

Clipped Surrogate Objective

The Actor is trained with a policy gradient method: its parameters θ are updated by stochastic gradient ascent on $J(\theta)$. By the policy gradient theorem [30], with V^π as a baseline that reduces variance without changing the gradient in expectation, this gradient can be written in advantage form:

$$\nabla_\theta J(\theta) \;\propto\; \mathbb{E}_\pi \left[A^\pi(s_t, a_t)\, \nabla_\theta \ln \pi_\theta(a_t \mid s_t) \right]. \tag{2.9}$$

The update makes actions with positive advantage more likely and actions with negative advantage less likely. Followed directly, however, this gradient is vulnerable to destructively large parameter updates: a single oversized gradient step can collapse the policy distribution irreversibly. PPO resolves this by optimizing a clipped surrogate objective that constrains how much the policy can change in a single update. Defining the probability ratio between the current and previous policy as:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \tag{2.10}$$

the Actor is trained to maximize the clipped surrogate objective $L^{\text{CLIP}}(\theta)$:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right], \tag{2.11}$$

where $\epsilon \in (0, 1)$ is the clipping hyperparameter (typically $\epsilon = 0.2$) and $\hat{A}_t$ estimates the advantage $A^\pi(s_t, a_t)$ from the collected rollout. It is computed with Generalized Advantage Estimation [26], an exponentially weighted average of multi-step

temporal-difference errors whose parameter $\lambda \in [0, 1]$ interpolates between a low-variance but biased one-step estimate and an unbiased but high-variance Monte Carlo one. The min operator selects the pessimistic (lower) bound between the clipped and unclipped objectives, preventing the ratio r_t from moving too far from 1 in either direction.

Combined Actor-Critic Loss

The Actor and Critic are trained jointly by optimizing a single combined loss function:

$$L^{\mathrm{PPO}}(\theta, \phi) = \hat{\mathbb{E}}_t \left[L_t^{\mathrm{CLIP}}(\theta) - c_1 \underbrace{\left(V_\phi(s_t) - V_t^{\mathrm{target}} \right)^2}_{L_t^{\mathrm{VF}}(\phi)} + c_2 \, S[\pi_\theta](s_t) \right], \tag{2.12}$$

where $L_t^{\mathrm{CLIP}}(\theta)$ denotes the per-timestep term inside the expectation of Eq. (2.11), $L_t^{\mathrm{VF}}(\phi)$ is the squared-error value function loss that trains the Critic to predict accurate returns, with the regression target $V_t^{\mathrm{target}} = \hat{A}_t + V_\phi(s_t)$ obtained by correcting the value predicted during the rollout with the estimated advantage, $S[\pi_\theta](s_t)$ is an entropy bonus that prevents the Actor from collapsing to a deterministic policy prematurely, and $c_1, c_2 > 0$ are weighting coefficients (typically $c_1 = 0.5$, $c_2 = 0.01$).

Algorithm 1 shows how these components fit together. Each iteration alternates between a rollout phase, in which the current policy is run in N environments simulated in parallel, and an optimization phase, in which the resulting batch of NT transitions is reused for K epochs of minibatch updates.

Algorithm 1 PPO training loop as used in this work, adapted from [27, Alg. 1].

Initialize Actor π_θ and Critic V_ϕ
for iteration $= 1, 2, \ldots$ **do**
    $\theta_{\mathrm{old}} \leftarrow \theta$
    for each of the N parallel environments, simultaneously **do**
        Run $\pi_{\theta_{\mathrm{old}}}$ for T timesteps, storing s_t, a_t, r_t and $\ln \pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)$
    end for
    Compute advantage estimates $\hat{A}_t$ and value targets V_t^{target} for the NT transitions
    for epoch $= 1, \ldots, K$ **do**
        for all minibatches of size $M \leq NT$ **do**
            Update θ and ϕ by gradient ascent on $L^{\mathrm{PPO}}(\theta, \phi)$, Eq. (2.12)
        end for
    end for
    Discard the collected transitions
end for

2.1.3 PAL Robotics and the TIAGo Pro Platform

PAL Robotics is a Barcelona-based company, founded in 2004, that designs and builds humanoid and mobile service robots for research and industry. Its catalogue

spans bipedal humanoids such as TALOS and Kangaroo, mobile bases, and the mobile manipulators of the TIAGo line, all delivered as open platforms running ROS on the robot's own onboard computer, which is what makes them suitable as research testbeds rather than closed products. The TIAGo mobile manipulator family was introduced in 2016 [20], conceived from the start as a modular platform whose arm, gripper and sensor configuration could be adapted to the needs of each laboratory. The **TIAGo Pro** is the latest member of this family, a mobile manipulator targeting research and applied development in manipulation and human-robot interaction [21], shown in Figure 2.2.

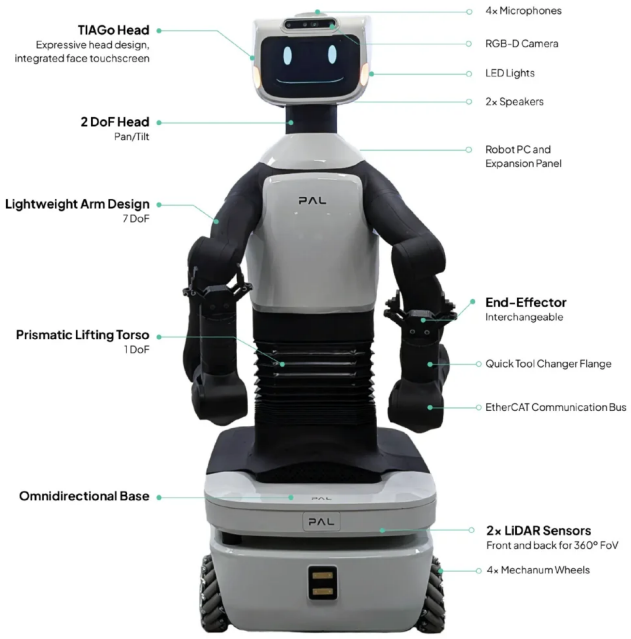


Figure 2.2. Overview of the TIAGo Pro mobile manipulator and its main hardware components [21].

According to the manufacturer's specifications [21], the TIAGo Pro comprises:

- **Mobile Base and Lifting Torso:** An omnidirectional base driven by four Mecanum wheels (3 DoF, maximum speed 1.5 m/s, indoor operation) and a prismatic torso lift with a 0.35 m stroke, giving an overall robot height of 1.20–1.50 m and a mass of approximately 96 kg.
- **Head Perception Suite:** An Intel RealSense D435 RGB-D camera mounted on a 2-DoF pan-tilt head, together with a four-microphone array.
- **7-DoF Arms:** Each arm has seven joints (four arm joints and a three-joint wrist), a reach of 0.96 m, a rated payload of 3 kg, and an ISO 9409-1 mounting flange; the reference configuration is dual-arm (2×7 DoF). Every arm joint is driven by a Series Elastic Actuator (SEA), whose compliant element reduces

the risk of injury during human-robot contact. The resulting kinematic chain is shown in Figure 2.3.

- **PAL Pro Gripper:** A single-DoF parallel gripper serves as the default end-effector.

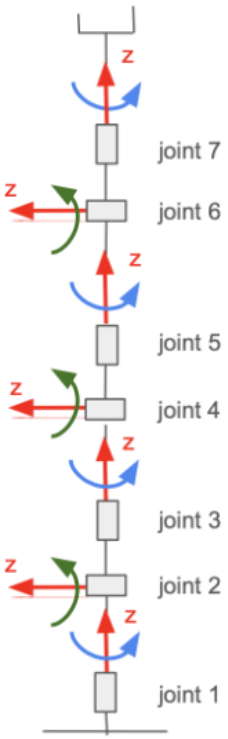


Figure 2.3. Kinematic structure of the 7-DoF TIAGo Pro arm, with the rotation axis z of each joint. The arm is the subsystem controlled by the policy learned in this work.

The TIAGo Pro is built on a software architecture fully compatible with the Robot Operating System 2 (ROS 2) [15], which is widely adopted in the robotics research community.

2.2 Related Work

2.2.1 Learning-Based Approaches to Robotic Manipulation

Traditional manipulation pipelines are assembled from explicit models: analytical Inverse Kinematics (IK), geometric motion planners and joint-level PID controllers. They perform well where the scene is structured and known in advance, and degrade as soon as it is not. Following Kroemer et al. [12], the shift toward learning-based methods can be justified on three grounds. The first is contact. Analytical models rely on rigid-body mechanics and simplified Coulomb friction, which fail to capture the non-linear, hybrid contact forces, the variations in surface friction and the compliance inherent to real manipulation tasks, whereas a learned policy adapts to

them directly through interaction experience. The second is generalization. A classical motion planner requires exact prior knowledge of the object geometry and must re-plan for every variation of the scene, while a learned policy acquires representations that generalize across object poses, shapes, mass distributions and workspace arrangements. The third is reactivity at execution time. Classical pipelines often decouple perception, motion planning and trajectory tracking into sequential stages, and because generating a collision-free trajectory or updating a task plan takes time, they tend to run in open loop between one planning cycle and the next. A learned policy instead needs a single fast inference per control step, which closes the loop at high frequency on the sensory observations and allows the robot to adapt to physical disturbances or moving targets as they occur.

Learning-based methods differ in the source from which the behaviour is acquired. *Imitation Learning* (IL) acquires it from an expert. In its most common form, Behavioral Cloning (BC), a policy is trained by supervised regression from observations to actions on a set of demonstrations. Modern architectures such as ACT [35] and Diffusion Policy [3] have significantly raised the quality of the imitated motions. BC nevertheless suffers from a structural limitation: the policy is trained on the state distribution of the expert but deployed under its own, so that any small prediction error moves the robot into states that the demonstrations do not cover, where further errors compound. This phenomenon is known as *covariate shift*. As a consequence, the performance of BC degrades with the task horizon, and large amounts of high-quality demonstrations are needed for the policy to generalize reliably.

Vision-Language-Action (VLA) models are also trained by imitation, but instead of learning a policy from scratch they start from a large vision-language model pretrained on text and images. RT-2 [36], for example, co-fine-tunes such a model on robotic trajectory data, encoding the actions directly as text tokens. The policy can thus draw on web-scale semantic knowledge and exhibits emergent capabilities, such as reasoning about object affordances and following natural language instructions for tasks never explicitly seen during robot training. The scale that enables this generalization also imposes a trade-off: inference through a multi-billion parameter transformer introduces a latency that limits the control frequency required for reactive, contact-rich manipulation tasks.

Reinforcement Learning, formalized in Section 2.1.1, is the approach adopted in this thesis. Unlike imitation learning it needs no demonstrations, since the behaviour is specified through a reward, and unlike VLA models it yields a compact policy that can run at the control frequency of the robot. Its costs are the number of interactions it requires, which confines training to simulation, and the resulting sim-to-real gap, both addressed in the next sections.

2.2.2 Reinforcement Learning for Robotic Manipulation: Overview

Reinforcement Learning entered robotic manipulation long before deep function approximation became practical. Early work operated on low-dimensional state and action spaces. This choice was dictated by the dominant constraint of the era: experience had to be collected on physical hardware. An algorithm was therefore useful only if it improved within a few hundred attempts. The common solution

was to shrink the search space: instead of learning a controller from scratch, the policy was a motion primitive with a handful of parameters, often initialized from a human demonstration and refined by episodic policy search. Peters and Schaal [23] illustrate the approach on a seven-degree-of-freedom anthropomorphic arm learning to hit a ball off a tee: a single demonstration teaches a rudimentary stroke, which the robot cannot execute accurately on its own, and a few hundred practice swings are then enough for the policy gradient updates to correct the hitting angle and produce a successful stroke.

Deep networks relaxed the curse of dimensionality that had confined earlier methods to small, hand-designed state spaces. As function approximators, they can represent policies and value functions over high-dimensional inputs and continuous, multi-joint action spaces. Levine et al. [13] demonstrated this at the extreme, training visuomotor policies that map raw camera images directly to the torques of a seven-joint arm. Larger spaces, however, take far more data to explore. One response was to collect data at industrial scale. QT-Opt [11] cast vision-based grasping as a reinforcement learning problem and trained a deep Q-function on over 580,000 grasp attempts collected on a fleet of real robots. The cost of this approach lay less in the algorithms than in the infrastructure: many robots, months of operation, and continuous supervision to keep them running.

The main lesson of this phase was that learning on real hardware at the scale deep RL requires remains beyond the reach of most laboratories.

The response was to move training into simulation and to compensate for the reduced fidelity with volume. GPU-accelerated environments such as Isaac Lab [16] and the MuJoCo-based mjlab [34] keep physics and policy on the same device and step thousands of instances in parallel, allowing a manipulation policy to be trained in minutes of computation on a single workstation, where the equivalent experience would have taken weeks of robot time. This regime favors on-policy algorithms such as PPO, whose stability matters more than sample efficiency once samples are cheap. In manipulation it delivered results that real-world data collection had never reached: Andrychowicz et al. [2] trained dexterous in-hand reorientation entirely in simulation and transferred it zero-shot to an anthropomorphic hand, and Handa et al. [5] later solved the same class of task from a camera-based estimate of the object pose.

Simulation, however, displaces the difficulty rather than removing it. A policy trained in a simulator is optimal with respect to that simulator's approximations, and it will exploit them wherever they are convenient: contact and friction models, actuator dynamics, sensor noise and control latencies all depart from their physical counterparts, so behaviour that succeeds in simulation can fail outright on hardware. This discrepancy is known as the *sim-to-real gap*, and it is what currently limits how much of the progress obtained in simulation reaches physical robots.

2.2.3 Sim-to-Real Transfer

A policy trained in simulation fails on hardware for reasons that are by now well catalogued: contact and friction models diverge from real surfaces, actuators exhibit dynamics, backlash and compliance that idealized joints do not, sensing is partial and noisy, and the control loop introduces latencies that the simulator is free of.

The techniques developed against these discrepancies attack them from different directions and are largely complementary.

The most widely adopted is domain randomization, which gives up on matching the simulator to reality and instead samples simulation parameters from distributions broad enough that reality is plausible under them. Tobin et al. [31] introduced the idea for the visual channel, training an object detector exclusively on low-fidelity renderings with randomized textures, lighting, camera placement and distractor objects, and obtaining a model accurate enough to localize objects for grasping on a real robot without a single real training image. Peng et al. [22] transposed the argument to the dynamics, randomizing masses, friction coefficients, damping and actuator delays while training a recurrent policy for a pushing task; because the parameters vary between episodes and the policy carries a memory of its own recent history, it learns to infer the dynamics it is facing and adapt, transferring to a physical arm with no real-world fine-tuning. Andrychowicz et al. [2] combined both channels at scale for in-hand reorientation. These methods share a common limitation: randomizing too widely yields conservative policies that trade task performance for robustness, so the choice of which parameters to randomize, and over which ranges, remains an empirical one.

A second direction improves the simulator rather than averaging over its errors. Hwangbo et al. [6] identified the actuators as the dominant source of mismatch for a quadruped driven by series-elastic elements, whose compliance and internal control loops resist analytical modeling; they replaced the idealized joint model with a network trained on measurements from the real hardware, mapping a short history of joint states and commands to the torque actually produced. With this component in place, policies trained only in simulation transferred to the physical robot and executed behaviors that hand-designed controllers had not achieved.

A third direction exploits the fact that simulation exposes quantities no sensor can measure, and uses them during training only. Pinto et al. [24] proposed the asymmetric actor–critic, in which the critic is given the true simulator state while the actor sees only what the real robot will have, so that value estimation becomes easier without making the deployed policy depend on privileged information.

2.2.4 Vision-Based Manipulation with RL

When manipulation must be guided by cameras, the design decision that organizes the literature is what the policy receives as input. Three approaches can be distinguished: raw images consumed by the policy itself, general-purpose visual encodings produced by a separately pre-trained network, and explicit task-relevant quantities extracted from the image before the policy sees anything.

The first approach trains perception and control as one network. Levine et al. [13] established it, optimizing a single model from pixels to torques, and it dominates today's imitation-learning systems introduced in Section 2.2.1: action-chunking transformers [35], diffusion policies [3] and vision-language-action models [36] all encode the image inside the policy. Nothing constrains what the network may extract, which is the argument in its favour, and the cost is data: these systems are trained on large sets of real demonstrations, where the images seen during training and deployment come from the same distribution.

The second approach separates representation learning from control. A vision encoder is pre-trained once and then frozen, and only a small policy head is trained per task. R3M [19] is pre-trained on egocentric human video and reported over 20% higher success than training the encoder from scratch across simulated tasks, and enabled learning real manipulation tasks from twenty demonstrations.

The third approach decides in advance what the policy is allowed to see: the image is reduced to a few task-relevant quantities before it reaches the policy. This can be done in two ways. The first is to discard appearance and keep only geometry, giving the policy depth images or point clouds, which a simulator reproduces far more faithfully than colour and texture. The second, more common, is to give the policy an estimate of the state of the object. Two prominent sim-to-real dexterity systems follow this route: a pose estimator is trained separately from the control policy, which then observes the estimated pose together with proprioception [2, 5]. The general argument in favour of this approach was given by Müller et al. [18]: a policy that is never exposed to raw perceptual input is insulated from the difference in appearance between training and deployment.

The choice among the three approaches follows mainly from where the training data comes from. When policies are learned from real demonstrations, the images seen in training and in deployment are alike, so they can enter the policy directly. When policies are learned in simulation and deployed without real data, the common choice is an input that simulation reproduces faithfully, such as depth or the object pose. The visual discrepancy is then confined to a separate perception component, which can be trained and evaluated on real images independently of the policy. This thesis belongs to this last case.

A second decision is where the camera is placed. In the *eye-in-hand* configuration it is mounted on the wrist or the end-effector of the robot, where it provides a close-up view of the grasp site and naturally follows the movement of the gripper. This requires dedicated hardware on the wrist, adds payload to the joint actuators, and restricts the field of view to the immediate vicinity of the end-effector, so that the broader context of the workspace is lost. In the *eye-to-hand* configuration the camera is fixed to the body of the robot, for example its head, or to an external support, and observes the workspace from a global viewpoint. It can rely on the standard onboard hardware, offers a wide view of the scene and is simpler to integrate, but during reaching and grasping the robot's own arm and end-effector inevitably occlude the line of sight to the object. The two decisions interact: with an eye-to-hand camera, a policy that takes images as its input receives uninformative observations precisely during the grasp, whereas a separate perception component can hold its last estimate through the occlusion and correct it once the object is visible again. The system presented in Chapter 3 combines an eye-to-hand camera with a separate perception component for this reason.

2.2.5 RL Works on TIAGo Family Platforms

Compared with the fixed industrial arms that dominate the manipulation literature, the TIAGo family appears in a small but varied body of reinforcement learning work. Albardaner et al. [1] addressed the sim-to-real gap directly, comparing the response of the TIAGo arm in Isaac Gym and in Isaac Sim against the real robot

and transferring a simple joint-space policy to hardware. Their treatment stays at the level of matching each simulator's PD joint controller to the position controller used on the platform, without examining the `ros_control` stack in any depth, the actuator behaviour underneath it, or any randomization of the simulated dynamics. Liu et al. [14] approached the same platform from the safety side, deploying safe RL on a physical TIAGo++ for high-dimensional manipulation in the presence of humans, and showing that learned policies can be constrained sufficiently to operate a high-DoF service manipulator in a shared workspace. Jauhri et al. [7] addressed the mobile side of the problem, learning where to place the base and whether to activate the arm to reach a 6D target, with the policy regularized by a reachability prior derived from the robot's workspace and transferred zero-shot to a physical TIAGo++; the same group later trained whole-body policies for articulated objects with offline reinforcement learning, reporting 80% success in bimanual drawer manipulation on the real robot without any teleoperated or real-world training data [8]. With respect to these studies, the work presented in this thesis addresses a problem that none of them covers: a complete vision-guided pick-and-place task on the TIAGo Pro, with a policy trained in simulation and deployed on the real robot using only its onboard camera.



(a) TIAGo **(b)** TIAGo++

Figure 2.4. The platforms used in the reinforcement learning studies discussed above: the single-arm TIAGo (left) and the dual-arm TIAGo++ (right).

Chapter 3

Methodology

The system proposed in this thesis is built from three components, each described in one section of this chapter. Section 3.1 covers the reinforcement learning framework: the simulated environment, the observation and action spaces, the reward design and the domain randomization under which the control policy is trained. Section 3.2 covers the perception pipeline that estimates the pose of the target object from the robot's head-mounted RGB-D camera. Finally, Section 3.3 describes the ROS 2 architecture that runs the two together on the robot at a fixed control rate.

3.1 Reinforcement Learning Framework

3.1.1 The mjlab Framework

All training experiments are conducted in **mjlab** [34], a recent open-source framework for robot learning that pairs GPU-accelerated simulation with composable environment definitions. It adopts the manager-based API introduced by Isaac Lab [16], and it runs the physics on MuJoCo Warp, the GPU implementation of MuJoCo [32].

The manager-based design is what shapes how an environment is written. Rather than implementing a monolithic environment class with a hand-written step function, the task is declared as a set of independent managers: an observation manager listing the terms that compose the observation vector, a reward manager holding the individual reward terms with their weights, a termination manager, an event manager for the perturbations applied at reset or at fixed intervals, and a command manager generating the goals an episode is conditioned on. Each term is a small function registered through a Python configuration object, so that a reward can be added, reweighted or removed, and a randomization can be switched on, without touching the training loop or the simulation code.

The performance side comes from MuJoCo Warp. Rigid-body dynamics, contact resolution and the integration step are evaluated as GPU kernels over a batch of environments held together in device memory, so that thousands of instances advance in parallel and their observations are passed to the policy network without leaving the GPU. Simulation and learning therefore stay on the same device for the entire training run, which is what makes the sample volume of on-policy algorithms such as PPO affordable on a single workstation.

3.1.2 Simulation Environment

In this framework a task is defined by its scene and by the terms registered in the managers listed above. The scene is deliberately minimal, since direct visual feedback is not part of the training, and mirrors the physical setup in which the policy is meant to operate: the robot stands in front of a table, a single object lies within reach of its right arm, and a delivery point is fixed to its right. The resulting environment is shown in Figure 3.1.

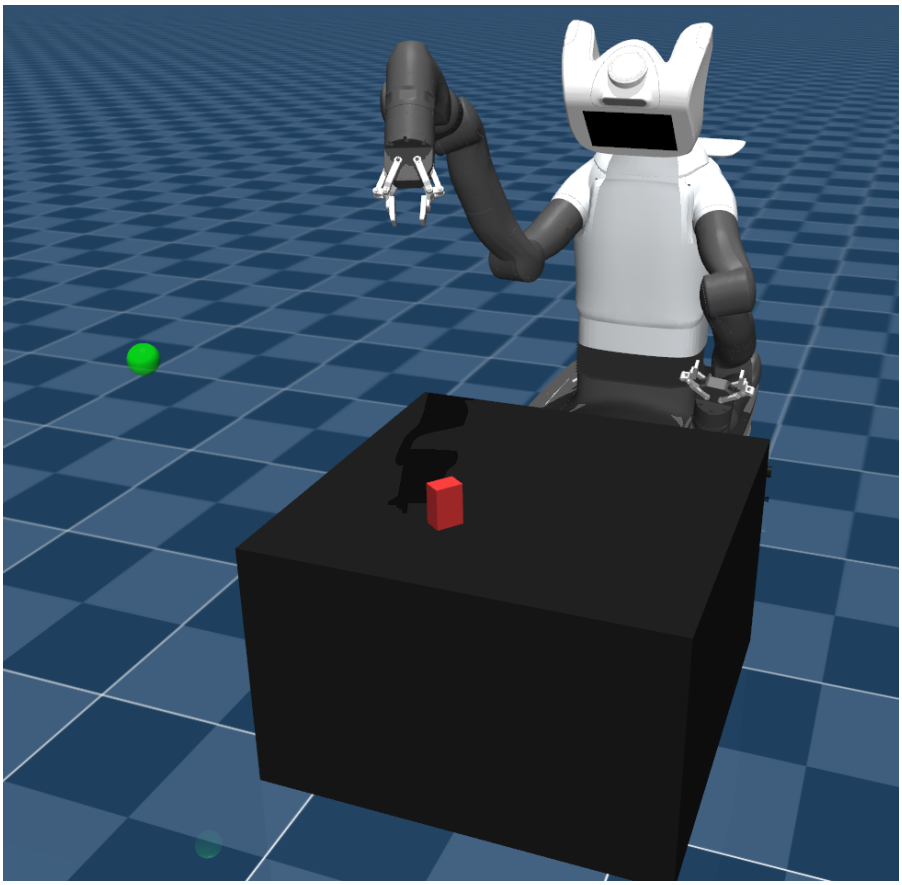


Figure 3.1. The simulated environment in **mjlab**, showing the TIAGo Pro robot, the fixed rectangular table, the red box representing the object to be manipulated, and the green sphere marking the 3D target coordinate.

It contains four entities:

- **TIAGo Pro robot**: the right arm and gripper are actuated, while the left arm is held in a fixed retracted pose.
- **Table**: a static support surface of $70 \times 70\,\text{cm}$ at a nominal height of $0.45\,\text{m}$.
- **Object**: a cuboid drawn in red.
- **Target coordinate**: a green sphere marking the 3D destination, used for visualization purposes.

The initial pose of the cube on the table and the delivery coordinate are resampled at every reset; the distributions they are drawn from are reported in Section 3.1.7, together with the rest of the randomization.

Two rates govern the environment. The physics solver runs at 200 Hz with a timestep of 5 ms, while the policy is queried at 50 Hz through a decimation factor of four, so that each action is held for four physics steps. An episode lasts at most $T = 5\,\mathrm{s}$, that is 250 policy steps, after which the environment resets and the scene is resampled. During training, 2048 such environments run in parallel.

3.1.3 PPO Implementation

The chosen algorithm is the PPO implementation provided by RSL-RL [28], the reinforcement learning library developed at ETH Zurich for robotics research and adopted as the default learning backend by both Isaac Lab and mjlab. Two of its characteristics shape how training behaves. The first is that it is GPU-resident: rollouts are stored and the updates computed on the device, without transferring batches to host memory, which is what allows the learner to keep up with the collection rate of the parallel simulator. The second is the adaptive learning rate: instead of following a fixed schedule, the step size is adjusted after every update according to the KL divergence measured between the old and the new policy, shrinking when the policy moves too far from its predecessor and growing when it barely moves. The nominal learning rate reported in Table 3.2, which collects all the training hyperparameters, is therefore only a starting point, and the target divergence of 0.01 is the quantity that actually governs the size of the updates. The batch of transitions collected at each iteration is split into four mini-batches and reused for five epochs before being discarded, with the advantages computed through Generalized Advantage Estimation [26].

Both the Actor and the Critic are multi-layer perceptrons with hidden dimensions $(512, 256, 128)$ and ELU activations. Observations are normalized online with an empirical running mean and standard deviation. The actor network outputs the mean $\boldsymbol{\mu} \in \mathbb{R}^8$ of a Gaussian policy. Its standard deviation is a single learnable parameter, shared across the action space and independent of the state, initialized at 1.0 and updated by the optimizer together with the network weights. The policy explores by sampling from $\mathcal{N}(\boldsymbol{\mu}, \sigma^2 I)$ during training, while at deployment the mean is taken directly and the policy is deterministic. Figure 3.2 illustrates the two networks.

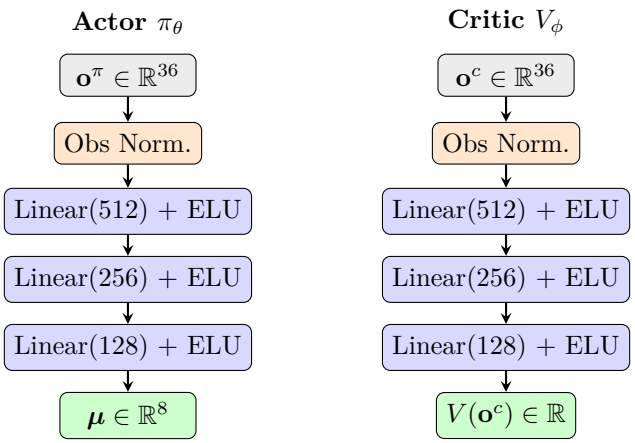


Figure 3.2. Actor and critic networks. The two share the same structure and observation layout, but we will later see that the actor receives noisy observations and the critic the clean simulator values.

3.1.4 Observation and Action Spaces

Two choices define the interface between the policy and the robot. The **observation space** is what the agent is allowed to know at each step: the subset of the environment state, on which the action is conditioned. The **action space** is what the agent is allowed to command. Both are design decisions and both are constrained here by the same requirement, that every quantity involved must exist on the real robot as well as in simulation.

For the work analysed in this thesis the actuated joints, hence the action space, were just those belonging to the right arm and the gripper. The joint controllers accept position references, so the policy operates in joint space, and at every control step the actor outputs an eight-dimensional vector, seven components for the arm joints and one for the gripper.

Before becoming actual position commands, each component returned from the policy is multiplied by a per-joint scale and the result is added to the *current* measured joint position to form the position target, so the policy commands displacements rather than absolute references. The scales are derived from the actuator specifications as $0.05 \cdot \tau_{\max}/k$, where $\tau_{\max}$ is the joint's effort limit and k the stiffness of its position controller, so that a unit action corresponds to the displacement that demands five percent of the available torque; the gripper scale is set to 0.01 m.

The *observation* is a 36-dimensional vector, assembled with the robot's own proprioception and the estimated pose of the object:

- **Arm joint positions** $\mathbf{q} \in \mathbb{R}^7$, expressed as displacements from their default values.
- **Arm joint velocities** $\dot{\mathbf{q}} \in \mathbb{R}^7$.
- **Previous action** $\mathbf{a}_{t-1} \in \mathbb{R}^8$.
- **Object position** in the base frame $\mathbf{p}_{obj} \in \mathbb{R}^3$.

- **Object yaw** $\in \mathbb{R}^2$, encoded as $[\cos\theta_{yaw}, \sin\theta_{yaw}]$ to avoid the discontinuity at the wrap-around.
- **Target position** $\mathbf{p}_{goal} \in \mathbb{R}^3$.
- **Gripper joint position** $g \in \mathbb{R}$.
- **End-effector position** $\mathbf{p}_{ee} \in \mathbb{R}^3$.
- **Contact signal** $c \in \{0, 1\}$, indicating simultaneous contact on both gripper fingers.
- **Reached flag** $c_{\text{reach}} \in \{0, 1\}$, which signals the transition from the carry phase to the release phase by turning to 1 once the goal coordinates are reached.

Every quantity in these vectors is either proprioceptive or produced by the perception pipeline of Section 3.2. Nothing in the observation requires a sensor the robot does not carry, which is the constraint that makes the policy deployable on the standard hardware configuration.

3.1.5 Reward Design

The reward is the only channel through which the task is communicated to the agent: at every step the environment returns a scalar, and the policy is optimized to maximize its discounted sum, so whatever behaviour maximizes that sum is the behaviour that will be learned. For a task as long as pick-and-place, the reward is usually *shaped*: it is written as a weighted sum of simpler terms, each responsible for one aspect of the behaviour and each producing a signal at every step, so that progress towards the goal is rewarded continuously rather than only at the end. The reward collected at step t is therefore

$$r_t = \sum_i w_i \, r_i(s_t, a_t), \tag{3.1}$$

where each r_i is one of the terms described below and w_i is its weight, reported in parentheses next to the name of the term and collected in Table 3.1. A positive weight makes the corresponding quantity something to seek, a negative one something to avoid, and the relative magnitudes decide which term prevails when two of them disagree.

Writing these terms is not a principled procedure for a contact-rich manipulation task. The terms below were determined following an iterative process: a behaviour observed in the rollouts suggested a term, the term was weighted by trial and error, and the resulting policy was inspected again, until the remaining failures were no longer attributable to the reward.

The resulting terms fall into two groups. **Task rewards** drive the sequence the task consists of, that is reaching the object, grasping it, lifting it, transporting it to the target and releasing it there; they describe *what* has to be achieved. **Behaviour rewards** constrain *how* it is achieved, suppressing the solutions that are nominally successful but useless on hardware: jittery or wasteful motion, approaches that strike the object from above, postures that hide the object from the camera, and contacts with the table or with the robot itself.

Task rewards

- **Reaching** ($w = +3.0$). Draws the end-effector towards the object. The term is one at contact and decays with the distance between the two, steeply within σ_r and more and more slowly beyond it, so that the gradient is concentrated where precision matters and distant configurations are treated as roughly equivalent:

$$r_{\text{reach}} = \text{clip}\left(1 - \tanh\left(\frac{\|\mathbf{p}_{ee} - \mathbf{p}_{obj}\|}{\sigma_r}\right),\ 0,\ 1\right), \quad \sigma_r = 0.15\,\text{m}. \tag{3.2}$$

- **Both-fingers contact** ($w = +1.0$). A binary term active when both fingertips register contact simultaneously, which rewards symmetric grasps over single-finger pushes.

- **Lifting** ($w = +1.0$). Activates once the object has left the table with both fingers in contact, and grows with the elevation Δz of the object. The growth is exponential in Δz and saturates at $\delta = 0.03\,\text{m}$: the term is worth $r_{\min} = 0.1$ the instant the cube leaves the surface, reaches $r_{\max} = 1.5$ once the cube is three centimetres clear of it, and stays there however high the cube is carried afterwards. Capping it prevents the policy from being paid for lifting ever higher, a behaviour that would compete with the goal-tracking term rather than support it:

$$r_{\text{lift}} = [\neg\text{on_table} \wedge \text{both_contact}] \cdot r_{\min} \cdot \left(\frac{r_{\max}}{r_{\min}}\right)^{\min(\Delta z,\,\delta)/\delta}. \tag{3.3}$$

- **Goal tracking** ($w = +5.0$). The term that drives the transport phase. It is active only while the object is off the table and held between both fingers, so that approaching the target without the cube earns nothing, and it decays with the distance between the object and the delivery point, reaching one when the two coincide. The decay scale is $\sigma_g = 0.50\,\text{m}$, wide enough to provide a gradient across the whole workspace:

$$r_{\text{goal}} = [\neg\text{on_table} \wedge \text{both_contact}] \cdot \left(1 - \tanh\left(\frac{\|\mathbf{p}_{obj} - \mathbf{p}_{goal}\|}{\sigma_g}\right)\right). \tag{3.4}$$

- **Post-reached end-effector stability** ($w = +1.0$). A binary reward for keeping the end-effector within 0.05 m of the goal after arrival, which prevents the arm from retracting before the object has been released:

$$r_{\text{stable}} = [\text{reached}] \cdot [\|\mathbf{p}_{ee} - \mathbf{p}_{goal}\| \leq 0.05\,\text{m}]. \tag{3.5}$$

- **Post-reached gripper opening** ($w = +5.0$). Encourages the gripper to open once the goal has been reached, which is what completes the task. The term is maximal when the gripper joint position g equals the reference aperture $g_{target} = 0.075\,\text{m}$, comparable to the mechanical limit of the gripper so that the maximum is attained with the fingers fully open, and decays exponentially as g departs from it, with $\sigma_{rel} = 0.025\,\text{m}$ setting how quickly:

$$r_{\text{release}} = [\text{reached}] \cdot e^{-|g - g_{target}|/\sigma_{rel}}. \tag{3.6}$$

Behaviour rewards

- **Gripper open during approach** ($w = +1.0$). Keeps the gripper open while the end-effector is still far from the object, preventing a premature closure that would block the approach. The term decays as the robot closes in:

$$r_{\text{open}} = \left(1 - e^{-\|\mathbf{p}_{ee} - \mathbf{p}_{obj}\|/\sigma_o}\right) \cdot \text{clip}\left(\frac{g}{g_{max}},\ 0,\ 1\right), \tag{3.7}$$

  with $\sigma_o = 0.02\,\text{m}$ and $g_{max} = 0.07\,\text{m}$, the aperture beyond which the gripper is treated as fully open.

- **Fingertip alignment** ($w = -5.0$). Penalizes the yaw misalignment between the gripper's squeeze axis and the cube, encouraging the fingertips to be parallel to the object's faces before contact. Because the cube is symmetric under 90° rotations, the relative angle between the two, $\Delta\theta_{\text{wrapped}}$, is wrapped to $[-\pi/4, \pi/4]$. The penalty scales with proximity and switches off once both fingers touch the object:

$$r_{\text{align}} = |\Delta\theta_{\text{wrapped}}| \cdot e^{-\|\mathbf{p}_{ee} - \mathbf{p}_{obj}\|/\sigma_a} \cdot (1 - c), \quad \sigma_a = 0.30\,\text{m}. \tag{3.8}$$

- **Table sliding** ($w = -5.0$). Penalizes lateral velocity of the object while it is still on the table, discouraging the agent from dragging the cube instead of lifting it.

- **Self and table collision** ($w = -2.0$). A binary penalty on contacts between the arm and either itself or the table.

- **Action rate** ($w = -0.01$). Penalizes consecutive action differences to promote smooth trajectories:

$$r_{\text{rate}} = \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2. \tag{3.9}$$

- **Joint occlusion** ($w = -0.5$). Penalizes configurations in which the first arm joint (`arm_right_1_joint`) drops more than 0.35 rad below its default position. This reward was added specifically for the first joint because it has been empirically identified as the main cause of the occlusion of the object from the head camera. The penalty grows exponentially with the violation:

$$r_{\text{occ}} = \text{clip}\left(e^{\,s \cdot \max(\phi_{\text{thr}} - \Delta q_1,\ 0)} - 1,\ 0,\ 10\right), \tag{3.10}$$

  with Δq_1 the displacement of that joint from its default position, $\phi_{\text{thr}} = -0.35\,\text{rad}$ and $s = 4$.

- **Joint torque** ($w = -5 \times 10^{-4}$). An L_2 regularizer on the applied torques, which favours energy-efficient motion.

The reward shaping strategy adopted for the task is summarized in Table 3.1, which lists every term with its weight and the condition under which it is active.

Table 3.1. Reward terms and weights, grouped by the role they play. The last two task rewards activate only after the reached flag becomes true.

Term	Type	Weight	Condition
Task rewards			
Reaching	Dense	$+3.0$	Always
Both-fingers contact	Binary	$+1.0$	Always
Lifting	Dense	$+1.0$	Off table and both fingers in contact
Goal tracking	Dense	$+5.0$	Off table and both fingers in contact
Post-reached stability	Binary	$+1.0$	After reached condition is met
Post-reached gripper open	Dense	$+5.0$	After reached condition is met
Behaviour rewards			
Gripper open	Dense	$+1.0$	Proximity-weighted, off after contact
Fingertip alignment	Dense	-5.0	Proximity-weighted, off after contact
Table sliding	Dense	-5.0	Object on the table
Self and table collision	Binary	-2.0	Always
Action rate	Dense	-0.01	Always
Joint occlusion	Dense	-0.5	Always
Joint torque	Dense	-5×10^{-4}	Always

3.1.6 Success and Termination Conditions

Training proceeds in episodes, and an episode can end in two different ways. It *terminates* when the environment reaches a state from which the task can no longer continue: the state is genuinely final, no further reward can be collected from it, and the return is computed as if the value of what follows were zero.

It is *truncated* when the episode is interrupted for reasons external to the task, typically a time limit, while the state itself is perfectly ordinary and the interaction could have continued. The distinction matters for the value target. Bootstrapping from the final state of a truncated episode preserves the estimate that the state deserves, whereas treating the interruption as a terminal state would teach the critic that running out of time is intrinsically bad, and the policy would learn to act as if the world ended at the horizon.

In this work an episode is truncated in two cases: when the time limit of 5 s is exhausted, and when the task has been executed successfully, that is when the agent has reached, grasped, lifted, transported and finally released the object at the delivery point. The condition combines two flags:

$$\text{success}_t = \text{reached}_t \wedge \neg\, \text{contact}_t, \tag{3.11}$$

where contact_t is the both-finger contact signal and reached_t marks that the object has been held within $d_{\text{succ}} = 0.03\,\text{m}$ of the target.

Two termination conditions proved necessary. The first fires when the cube falls below a height of 0.1 m, that is when it has left the table and is on the floor: nothing the policy does afterwards can recover the object, so the remaining steps would only accumulate the reward of an arm moving above an empty table and would teach the critic to value states from which the task is already lost. The second fires when a NaN value appears anywhere in the simulation state. This is a numerical safeguard

rather than a task condition: contact solvers occasionally produce an invalid state, and without the check a single corrupted transition would enter the batch and propagate through the gradient update, destabilizing the policy.

3.1.7 Bridging the Sim-to-Real Gap

A policy trained in simulation is optimized for the simulator, and the real robot differs from the simulator in several respects. The table height and the robot base pose deviate from the modelled values. The joint encoders have small calibration errors, and friction varies from joint to joint. The object to be picked is generally not the cube used in training, and its pose is not read from the simulator state but estimated by a perception pipeline with an error of a few millimetres. A policy trained without accounting for these imperfections is unlikely to transfer.

This gap is addressed with the *domain randomization* techniques, introduced in Section 2.2.3. Instead of being fixed, each of the quantities above is sampled from a range chosen to contain the value expected on the real robot, in most cases at the start of every episode. The policy is thus trained on a distribution of environments rather than a single one, which discourages overfitting to any particular instance. For example, because the table height changes between episodes, the policy must rely on the observed object pose; because the encoder readings are perturbed, it must correct its motion in closed loop rather than reproduce a memorized trajectory.

The randomizations fall into four groups, described below. The first two concern the environment: the scene configuration at the start of each episode and the physical properties of the manipulated object. The third perturbs the observations available to the policy, and introduces the asymmetric critic. The fourth covers systematic differences between individual robots, which are sampled once when the environment is created rather than at every reset.

Configurational randomization

At the beginning of every episode the scene is rebuilt by drawing each of the following quantities from a uniform distribution over the stated range:

- **Robot placement**: the origin of the robot base frame is moved in the world by $\mathcal{U}(-0.05, 0.05)$ m along x and y and turned by $\mathcal{U}(-0.1, 0.1)$ rad in yaw. This randomizes the pose of the robot with respect to the table.
- **Arm initial configuration**: the arm starts from one of two nominal poses, selected with equal probability, and in both cases each joint value is perturbed by adding an offset sampled from $\mathcal{U}(-0.1, 0.1)$ rad. The first of the two nominal poses holds the arm above the table, as at the beginning of a task execution; the second holds it over the release area, as at the end of one. Training from both helps the policy to execute the task repeatedly, approaching a new object on the table from the posture in which the previous one was dropped.
- **Gripper initial position**: the gripper's joint initial value is perturbed by an offset drawn from $\mathcal{U}(-0.01, 0.01)$ m around its nominal opening which corresponds to the fingers at half the maximum aperture.

- **Table height**: sampled from $\mathcal{U}(-0.1, 0.1)$ m around the standard 0.45 m.
- **Cube pose**: the cube position is sampled from $\mathcal{U}(-0.275, 0.275)$ m along each of x and y about the table centre, located at $x = 0.50$ m, $y = 0.00$ m in the robot frame, and its yaw from $\mathcal{U}(-45°, 45°)$. The position range has been chosen to cover the entire table surface.
- **Delivery point**: the 3D target is drawn uniformly from $x \in [0.62, 0.77]$ m, $y \in [-0.766, -0.6]$ m and $z \in [0.55, 0.65]$ m in the same frame.

Together, these randomizations widen the region of the state space that the policy explores during training. In each episode the cube appears at a different position and orientation on the table, the arm starts from one of two configurations, and the table height and the robot base pose change. The policy must therefore perform well over a whole region of the state space, not just along a single trajectory.

Physical parameter randomization

The policy is expected to handle cuboids with different physical characteristics blindly, that is, without receiving any explicit information about them. The dimensions and the inertial properties of the object are therefore randomized:

- **Cube dimensions**: the width along each of x and y is sampled between 2 and 5 cm, and the height along z between 4 and 8 cm.
- **Cube mass**: the mass is sampled between 10 and 100 g.

The friction coefficient of the object is not randomized. The fingers of the TIAGo Pro gripper are coated with high-friction rubber, and the objects used in the real-world tests were not particularly slippery, so friction was not expected to be a significant source of sim-to-real mismatch.

Observation noise and the asymmetric critic

On the real robot, every quantity observed by the policy is affected by measurement error. To prevent the policy from relying on exact measurements, uniform noise is added to its observations at every time step:

- **Arm joint positions**: $\mathcal{U}(-0.02, 0.02)$ rad.
- **Arm joint velocities**: $\mathcal{U}(-0.05, 0.05)$ rad/s.
- **Gripper position**: $\mathcal{U}(-0.003, 0.003)$ m.
- **End-effector position**: $\mathcal{U}(-0.01, 0.01)$ m.
- **Object position**: $\mathcal{U}(-0.01, 0.01)$ m.
- **Object yaw**: $\mathcal{U}(-0.05, 0.05)$ rad.

The noise ranges were chosen to reproduce as closely as possible the noise encountered on the real hardware. The noise on the joint, gripper and end-effector measurements accounts for the encoder and calibration errors of the real arm, while the noise on the object position and yaw reproduces the error of the perception pipeline, whose accuracy is characterized in Chapter 4.

The noise is applied to the observations of the actor only. In actor-critic methods the two networks play different roles: the actor is the policy that is eventually deployed on the robot, whereas the critic estimates the value function and is used only during training, to compute the advantages that drive the policy update. Since the critic is discarded after training, its inputs need not be limited to what can be measured on the real robot. This is the idea behind the asymmetric actor-critic scheme of Pinto et al. [24].

In this work the asymmetry is limited to the noise. Corrupting the inputs of the critic would bring no benefit, since it would only make its value estimates less accurate, which in turn makes the policy updates noisier. Keeping the critic observations clean gives the actor a more reliable learning signal, while the actor itself still learns to act on the degraded observations it will receive on the real robot.

Systematic hardware discrepancies

Every physical robot may be affected by manufacturing and assembly errors, which introduce persistent biases in its measurements. Unlike observation noise, these errors do not vary from step to step: they are therefore sampled once, when the environment is created, and kept fixed afterwards. Since the task is not highly dynamic, only the following quantities were randomized:

- **Encoder calibration offset**: a constant bias sampled from $\mathcal{U}(-0.01, 0.01)$ rad is added to the zero position of each joint.
- **Joint friction loss**: the friction of each actuator is offset from its nominal value by a term sampled from $\mathcal{U}(-0.005, 0.005)$ N·m.

3.1.8 Training Summary

Table 3.2 summarizes the hyperparameters with which the policy is trained.

Table 3.2. PPO training hyperparameters.

Hyperparameter	Value
Parallel environments	2048
Rollout steps per environment	24
Mini-batches per update	4
Learning epochs per iteration	5
Learning rate	1×10^{-3} (adaptive KL schedule)
Discount factor γ	0.99
GAE λ	0.95
Clipping parameter ϵ	0.2
Entropy coefficient c_2	0.005
Value loss coefficient c_1	1.0
Maximum gradient norm	1.0
Target KL divergence	0.01
Training iterations	15,000

3.2 Visual Perception Pipeline

As discussed in Sections 1.4 and 2.2.4, this work adopts a decoupled approach between reinforcement learning and perception. As seen in Section 3.1.4, the policy must therefore be fed with an estimate of the object's 3D coordinates in the robot base frame, along with a yaw value representing its orientation.

The pipeline presented in this section computes this estimate from the RGB and depth streams of the head-mounted Intel RealSense D435. It consists of a first object detection step through YOLO, followed by classical, non-learning-based operations on the resulting region of interest.

3.2.1 Pipeline Overview

The pipeline runs asynchronously, triggered by incoming camera frames, in four stages:

1. **2D detection**: a single-stage detector locates the object in the RGB image and returns a bounding box.

2. **Colour segmentation**: the crop inside the box is converted to HSV, and a dominant colour reference estimated from its centre is used to separate the object's pixels from shadows, table texture and background clutter.

3. **Depth registration and back-projection**: the resulting binary mask is applied to the aligned depth image, invalid and outlying depths are rejected, and the surviving pixels are back-projected into 3D.

4. **Pose estimation**: the projected point cloud is transformed into the robot base frame, where an oriented bounding box fitted in the horizontal plane yields the centroid (X, Y, Z) and the yaw angle θ.

3.2.2 YOLO Object Detection

You Only Look Once

The first stage uses YOLO (You Only Look Once) [25], a fast single-stage detector. Unlike traditional methods that crop and classify candidate regions in multiple steps, YOLO evaluates the entire image in a single forward pass. The network divides the input into a spatial grid; if an object's center falls inside a cell, that cell is responsible for detecting it. In one shot, the model predicts bounding boxes $[u_{\min}, v_{\min}, u_{\max}, v_{\max}]$, confidence scores $C \in [0, 1]$, and class labels across all cells simultaneously, enabling high-speed real-time detection.

The detector used in this work is YOLO26n [9], the smallest and fastest model of the YOLO26 family.

Dataset

The detector is trained on a custom single-class dataset, which merges three public collections of cubes and other graspable objects with synthetic images rendered from the head camera of the TIAGo Pro in the Gazebo simulation. The table texture and the colour of the object in the Gazebo simulated environment are randomized across the images to make the detector more robust to lighting, shadows and colour shifts. In total, the merged dataset contains 4,552 training and 888 validation images.

The dataset is deliberately not limited to cubes. Although the policy is trained exclusively on cuboids, it never receives the image itself, only the position and yaw estimated by the pipeline. For this reason it should be possible to grasp any reasonably shaped object as long as it is recognized by the detector.

Figure 3.3 shows three representative samples of the dataset: on the left an image from the Gazebo simulated dataset, followed by two images taken from the public collections.

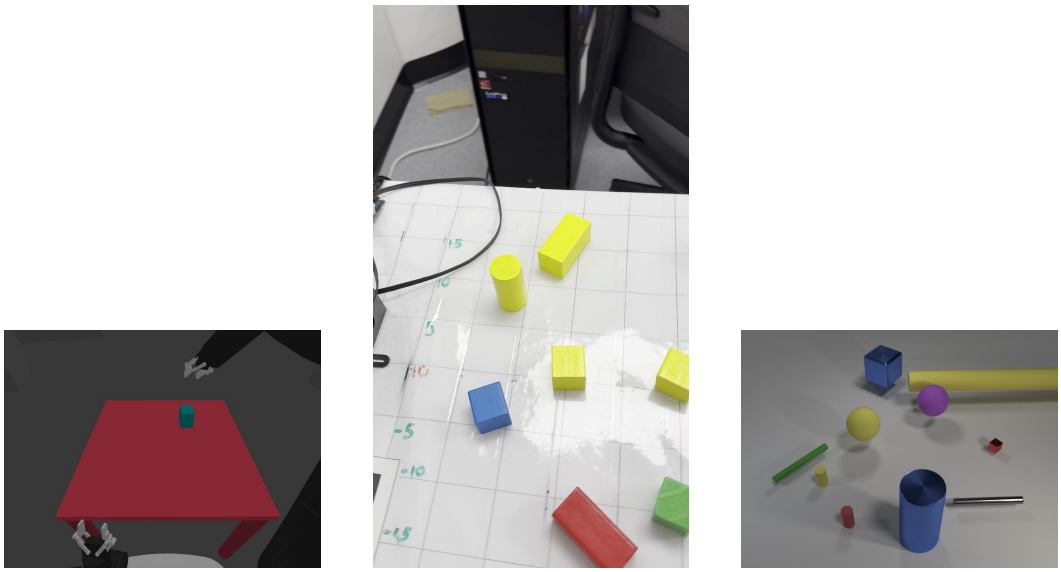


Figure 3.3. Representative images from the unified single-class dataset, showing the range of viewpoints, scales and textures across the simulated and real domains.

Training Process and Validation Metrics

Training does not start from scratch: it takes advantage of the YOLO26n weights pretrained on COCO, a large public dataset of everyday scenes annotated with bounding boxes for 80 common object categories, and fine-tunes them on the dataset described above. In this way the detector reuses the general visual features already learned on COCO, instead of having to learn them from a few thousand images. Training runs for 250 epochs with a batch size of 16 and images resized to 640×640 pixels. The learning rate starts at 0.01 and decays linearly.

With 4,552 images the dataset can be considered small, and may poorly reproduce the conditions of deployment: the public collections were photographed with other cameras, viewpoints and lighting, while the simulated images are rendered with flat colours and without sensor artefacts. A detector trained on these images as they are would tend to overfit on them, causing failures once deployed on the real robot. *Data augmentation* addresses this problem by corrupting the training images in various ways, so as to make the detector more robust, for example, to changes in illumination, to the blur caused by the vibration of the head, and to the different viewpoints from which the table is observed. The following distortions were adopted:

Illumination and colour. HSV jittering of $\pm 2\%$ in hue, $\pm 70\%$ in saturation and $\pm 40\%$ in brightness, so that the detector adapts to the lighting of the deployment room.

Geometry. Rotations of up to $\pm 30°$, scale changes of $\pm 50\%$, translations of $\pm 15\%$, mild perspective distortion and horizontal flips with probability 0.5.

Motion blur. Gaussian blur applied to 35% of the images with kernel size $k \in \{3, 5, 7\}$ and $\sigma \in [0.8, 2.5]$, modelling the blur induced by head trembling produced by the arm during task's execution.

The detector is evaluated on the validation images using four standard metrics. The first two are *precision*, the percentage of predicted boxes that are correct, which indicates how well the model avoids false alarms, and *recall*, the percentage of ground-truth objects that are successfully detected, which indicates how rarely the model misses a target. To decide whether a predicted box matches the ground truth, their spatial overlap is measured with the *Intersection over Union* (IoU), that is the area of their intersection divided by the area of their union, and a prediction is counted as correct only if its IoU exceeds a given threshold.

Because precision and recall shift with the confidence score required to accept a prediction, the *average precision* (AP) summarizes this trade-off as the area under the precision-recall curve. The *mean average precision* (mAP) then averages the AP across all classes, here a single one. Finally, mAP_{50} and $\text{mAP}_{50\text{-}95}$ evaluate the model at different levels of strictness: mAP_{50} uses a lenient IoU threshold of 0.5, accepting predictions that roughly cover the ground truth, whereas $\text{mAP}_{50\text{-}95}$ averages the results over ten increasingly strict thresholds, from 0.5 to 0.95, and therefore rewards tightly fitted boxes.

Table 3.3 reports the validation performance of the trained model.

Table 3.3. Validation metrics of the trained YOLO26n detector.

Model	Epochs	Precision (%)	Recall (%)	mAP_{50} (%)	$mAP_{50\text{-}95}$ (%)
YOLO26n	250	98.48	98.46	99.37	82.24

Overall, the detector reaches high scores on all four metrics: false detections and missed objects are both rare, and almost every object is localized at least roughly. The lower value of $mAP_{50\text{-}95}$ is expected, since its stricter thresholds penalize even small misalignments of the box edges, and it still indicates boxes that fit the objects closely. To visualize these results, Figure 3.4 shows the output of the trained detector on the same dataset images presented in Figure 3.3.

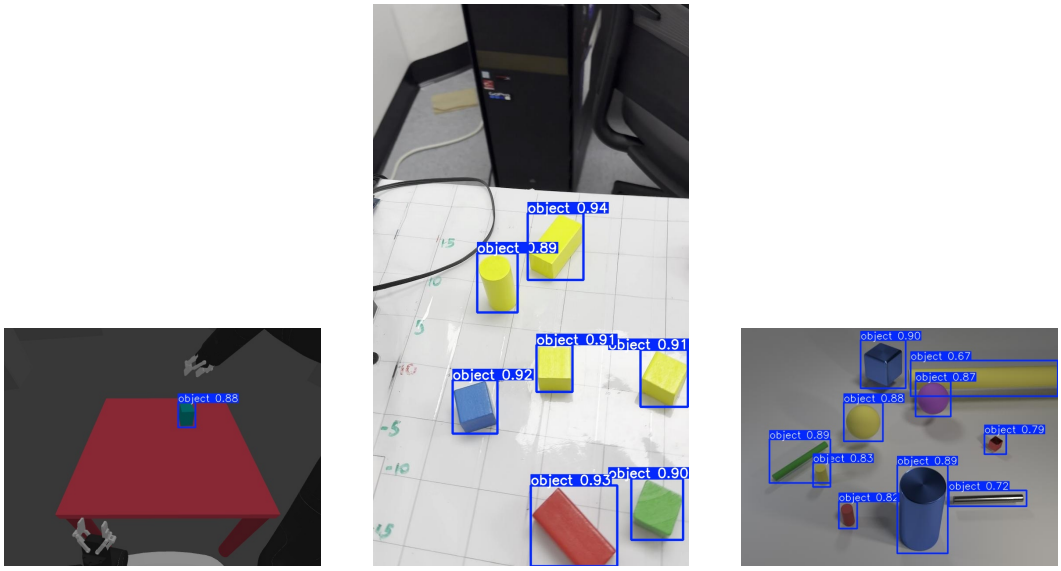


Figure 3.4. Detections on the sample images of Figure 3.3, with predicted boxes and confidence scores.

Validation on the Real Robot

The trained detector was finally tested by deploying it on the real robot, where it achieved the desired results. Figure 3.5 shows a frame of the head camera taken during deployment, in which a green bounding box tightly encloses the object.

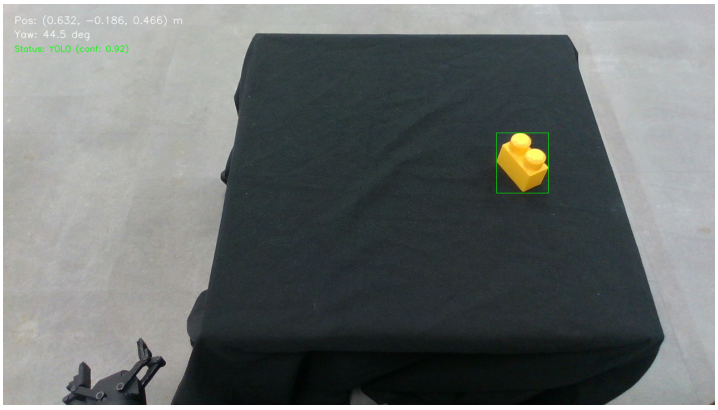


Figure 3.5. Detection during deployment: the target is isolated from the head-camera stream with its bounding box and confidence score.

3.2.3 Colour Segmentation of the Region of Interest

Once an object has been detected, the region of interest (ROI), that is the area of the image inside the bounding box returned by the detector, is processed to extract a mask that covers only the object, removing the background and any other irrelevant detail. This step is necessary because the box inevitably includes part of the table around the object, which could corrupt the computations that will be done in the next stages. The object is separated from the rest of the ROI on the basis of its colour.

For this reason the RGB ROI is converted to the HSV space, which describes each pixel by its hue (the colour itself), its saturation (how vivid the colour is) and its value (how bright it is). Assuming the object to be of a single colour and to occupy most of the central region of the bounding box, its dominant colour is sampled from the central 60% of the crop, so that the pixels near the edges of the box, which mostly belong to the table, do not affect it.

Saturation and value are ordinary scales, so their references are simply the medians of the corresponding channels over this central region. Hue, instead, is circular: the colours follow one another from red through yellow, green and blue and back to red, so that both ends of the range $[0, 180)$ in which OpenCV encodes it correspond to red. An ordinary average would therefore fail. For this reason the reference hue is computed as a circular mean: each hue value is treated as a direction on a circle, the directions are averaged, and the direction of the result is taken as the dominant hue. The raw hue values are first converted to angles,

$$H_{rad} = H_{raw} \cdot \frac{\pi}{90}, \tag{3.12}$$

then averaged as unit vectors,

$$H_{dom_rad} = \operatorname{atan2}(\operatorname{mean}(\sin H_{rad}),\ \operatorname{mean}(\cos H_{rad}))\,, \tag{3.13}$$

and the resulting angle, wrapped to $[0, 2\pi)$ when negative, is finally mapped back to the OpenCV range:

$$H_{dom} = H_{dom_rad} \cdot \frac{90}{\pi}. \tag{3.14}$$

The reference colour $(H_{dom}, S_{dom}, V_{dom})$, where S_{dom} and V_{dom} denote the two medians, is then used to decide, pixel by pixel, whether each pixel of the crop belongs to the object. The result is a binary mask M made of white pixels (255) representing the object and black pixels (0) elsewhere. The pixel at column u and row v is assigned to the object only if its hue, saturation and value are all close enough to the reference, and if it is neither too grey nor too dark:

$$M(u,v) = \begin{cases} 255 & \text{if } \Delta H < \tau_H \wedge \Delta S < \tau_S \wedge \Delta V < \tau_V \\ & \quad \wedge\, S(u,v) > 20 \wedge V(u,v) > 20, \\ 0 & \text{otherwise.} \end{cases} \tag{3.15}$$

Here ΔH, ΔS and ΔV measure how far the pixel is from the reference on each of the three scales, and τ_H, τ_S and τ_V are the tolerances allowed on each of them. For saturation and value the distance is the absolute difference,

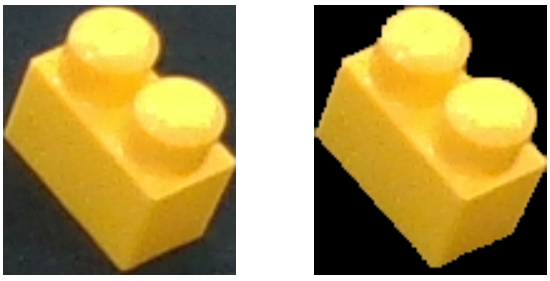
$$\Delta S = |S(u,v) - S_{dom}|, \qquad \Delta V = |V(u,v) - V_{dom}|, \tag{3.16}$$

whereas for hue, which is circular, it must be measured along the shorter of the two ways round the circle:

$$\Delta H = \min(|H(u,v) - H_{dom}|,\ 180 - |H(u,v) - H_{dom}|). \tag{3.17}$$

The last two conditions of the rule discard the pixels that are very grey or very dark: such pixels have almost no colour, so their hue is essentially noise and could match the reference by accident.

The mask obtained in this way is then slightly corrected by filling the small holes left inside the object, for instance by a reflection. Figure 3.6 shows an example of the result, in which the pixels retained by the mask are displayed in their original colour and the discarded ones in black.



(a) RGB crop. **(b)** Masked crop.

Figure 3.6. Colour segmentation: the crop extracted from the detected box (left) and the same crop after masking, in which only the pixels that pass the colour test are kept (right).

3.2.4 Depth Filtering and Back-projection

Once the object's pixels have been isolated in the RGB ROI, the next step works on the depth values associated with them. Since the RGB and depth images may differ in resolution, the coordinates of the ROI cannot be applied to the depth image

directly, and the two must first be aligned. This is done by computing the scaling factors

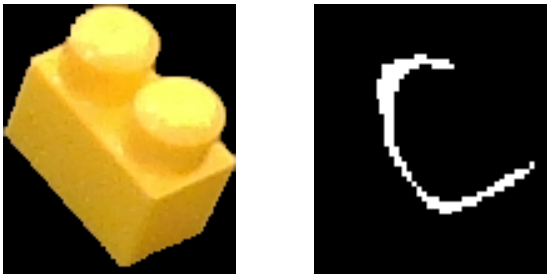
$$s_x = \frac{w_{depth}}{w_{rgb}}, \qquad s_y = \frac{h_{depth}}{h_{rgb}}, \tag{3.18}$$

which are used to rescale both the ROI coordinates and the camera intrinsics (f_x, f_y, c_x, c_y), so that all subsequent geometry is expressed consistently in depth-image pixels.

Depth values are then read only at the pixels retained by the colour mask. Since depth images tend to be noisier than RGB ones we discard invalid and out-of-range readings. Then the median depth Z_{median} of the survivors is computed, and a tight tolerance band rejects every pixel whose depth deviates from it by more than $\delta = 2.5\,\text{mm}$:

$$|Z_{(u,v)} - Z_{median}| < 0.0025\,\text{m}. \tag{3.19}$$

This strict design choice has proved necessary to filter out possible background pixels that could have passed the first RGB filtering operation. This was also possible due to the nature of the objects that the robot is supposed to work with: cuboids, in fact, present a regular structure and their pose could be described by just a small longitudinal section of them.



(a) Colour mask. **(b)** Depth inliers.

Figure 3.7. Depth filtering: the colour mask selects candidate pixels (left), and median-based outlier rejection retains the object's depth layer (right).

The surviving pixels are then back-projected through the pinhole model. For a pixel (u, v) with depth Z, the coordinates in the camera optical frame are

$$X = \frac{(u - c_x) \cdot Z}{f_x}, \qquad Y = \frac{(v - c_y) \cdot Z}{f_y}, \tag{3.20}$$

applied to the whole mask as a single vectorized operation, which yields the point cloud $\mathcal{P}_{cam}$ of the visible object surface.

3.2.5 Pose and Yaw Estimation

Before the final pose can be extracted, the point cloud must be expressed in the base frame of the robot, since this is the frame in which the policy was trained to receive the coordinates of the object. The cloud is therefore transformed with the camera-to-base transform,

$$\mathcal{P}_{robot} = \{T^{robot}_{camera} \cdot \mathbf{p}_{cam} \mid \mathbf{p}_{cam} \in \mathcal{P}_{cam}\}, \tag{3.21}$$

and projected onto the horizontal plane,

$$\mathcal{P}_{xy} = \{(x, y) \mid (x, y, z) \in \mathcal{P}_{robot}\}. \tag{3.22}$$

Finally, a minimum-area rectangle is fitted to $\mathcal{P}_{xy}$, as shown in Figure 3.8. Its centre gives the horizontal coordinates (X, Y) of the object, and the direction of its edges gives the orientation of the object in the base frame, hence the yaw. The vertical coordinate Z is taken as the midpoint between the highest and the lowest point of $\mathcal{P}_{robot}$. The four ordered corners of the rectangle are denoted $\mathbf{p}_k$.

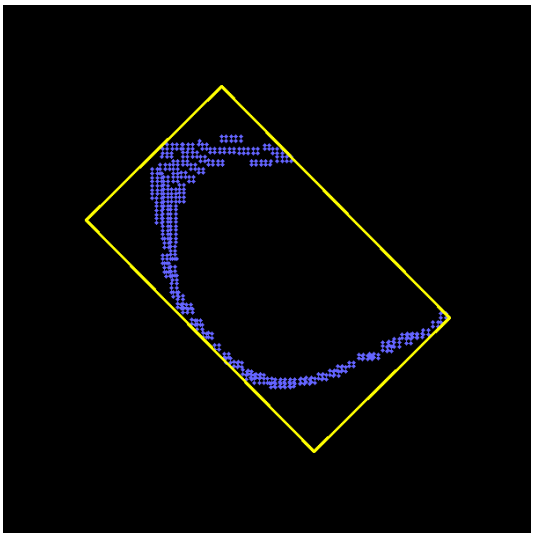


Figure 3.8. Minimum-area rectangle fitted to the point cloud projected on the horizontal plane of the base frame.

The yaw is obtained from the edges of the rectangle. Two adjacent edges, which are perpendicular to each other, are described by the vectors

$$\mathbf{v}_1 = \mathbf{p}_1 - \mathbf{p}_0, \qquad \mathbf{v}_2 = \mathbf{p}_2 - \mathbf{p}_1, \tag{3.23}$$

and the angle that each of them forms with the x axis of the base frame is $\theta_i = \operatorname{atan2}(v_{i,y}, v_{i,x})$. An edge, however, has no preferred direction: traversing it one way or the other describes the same side of the object, so its angle is only meaningful up to half a turn. Each θ_i is therefore brought into the interval $[-\pi/2, \pi/2)$ by adding or subtracting π.

After this step the two angles differ by exactly $90°$, and for an object with a square footprint both describe its orientation equally well, since a cube rotated by a quarter turn looks the same. The estimate is taken as the one of smallest absolute value, that is the angle of the edge closest to the x axis, which lies in $[-\pi/4, \pi/4]$: this angle is chosen because the policy is trained on yaw values belonging to this range, as described in Section 3.1.7.

3.3 ROS 2 Deployment Architecture

The trained policy and the perception pipeline are run together on the robot by a ROS 2 system [15], described in this section. Section 3.3.1 presents the nodes that

compose it and the data they exchange, from the sensors to the joint controllers. Section 3.3.2 describes the node that runs the perception pipeline and the filtering applied to its estimates. Section 3.3.3 explains how a grasp is detected without a dedicated sensor, and how this information is used to correct the observation given to the policy. Section 3.3.4 describes the three control nodes in more detail. Finally, Section 3.3.5 discusses the timing of the loop.

3.3.1 Nodes Architecture

The system consists of five nodes, shown in Figure 3.9. The nodes can be divided into Perception Nodes, denoted in cyan, and Control Nodes, denoted by the green colour. The Perception Nodes are written in Python, while the Control Nodes have been defined in C++, which gives the best performance when dealing with strict control execution times.

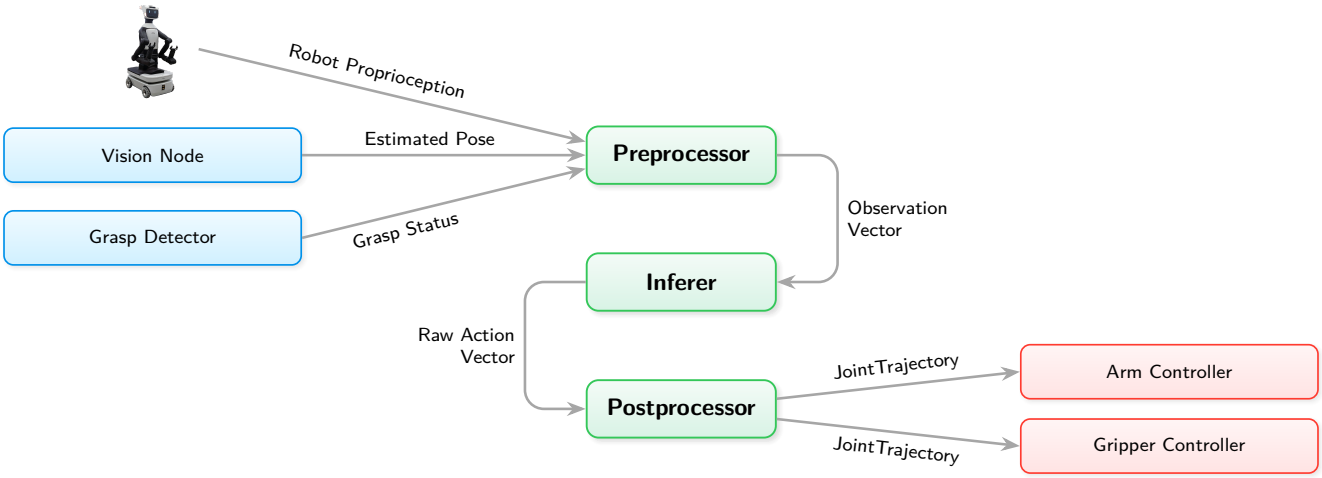


Figure 3.9. Overview of the ROS 2 control system. The perception nodes, together with the proprioception of the robot, provide the preprocessor with the information needed to assemble the observation vector. The inferer then evaluates the policy, and the post-processor converts the resulting actions into joint commands for the arm and gripper controllers.

1. **Vision node**. Runs the pipeline detailed in Section 3.2 on the RGB-D stream and publishes the filtered predicted object pose.

2. **Grasp detector**. Monitors the gripper motor current and finger position and publishes a binary contact flag.

3. **Preprocessor**. It builds what the policy sees. At every control step it collects the most recent value of each quantity and arranges them in the observation vector, in the same order and with the same conventions used during training.

4. **Inferer**. It runs the trained policy. At every control step it takes the most recent observation vector, evaluates the actor network on it and publishes the resulting eight-dimensional action, seven components for the arm joints and one for the gripper.

5. **Postprocessor**. It turns the action into commands that the robot can execute. Each time a new action arrives, it multiplies it by the scale of the corresponding joint and adds the result to the current measured position of that joint. The resulting position targets are then sent to the arm and gripper controllers.

3.3.2 Vision Node

The perception pipeline is realized as an asynchronous Python node. High-frequency sensor subscriptions use a best-effort QoS profile with history depth one, so that the node always processes the newest frame instead of accumulating a queue when a frame takes longer than usual.

The node subscribes to three topics: the camera info topic, from which the intrinsic parameters are read, the colour image stream and the depth image stream. It publishes the estimated pose, together with visualization markers and an annotated image stream used for monitoring on RVIZ.

The pose estimates oscillate around the true value from one frame to the next, so they are filtered before being published. The filter used is an *exponential moving average* (EMA). At every new frame k, the filtered value $x^{(k)}$ is computed as a weighted sum of the new raw estimate $z^{(k)}$ and of the previous filtered value $x^{(k-1)}$:

$$x^{(k)} = \alpha\, z^{(k)} + (1 - \alpha)\, x^{(k-1)}. \tag{3.24}$$

The smoothing factor $\alpha \in [0, 1]$ sets how much the new estimate counts. With $\alpha = 1$ the output is the raw estimate and nothing is filtered; the smaller α is, the less a single frame can move the output, which becomes smoother but also slower to follow a real change. Since the object stands still on the table and its pose only needs to be estimated during the approach and grasp phases, this delay is negligible.

The position and the yaw are handled as follows:

- **Position**: each of the three coordinates is filtered separately, with $\alpha_{pos} = 0.20$.
- **Yaw lock**: in the preliminary tests on the robot, the yaw estimate proved less stable than the position estimate, and its quality degraded progressively during the approach, as the arm began to occlude the object. An EMA filter alone did not give satisfactory results, so a simpler solution was adopted. Since the orientation of the object cannot change until the gripper touches it, the yaw estimates are recorded during the first 3 s of the task, and their median is then kept fixed for the remainder of the approach and grasp phases.

Finally, the object may fail to be detected in some frames during the execution of the task, for instance when the arm hides it from the camera. In that case the node keeps publishing its most recent estimate, so that the policy continues to receive a valid object pose until the detection is recovered.

3.3.3 Grasp Detection and Kinematic Override

The TIAGo Pro gripper has no dedicated force or tactile sensor, so contact is inferred from the motor current combined with the finger position. A hysteresis rule, shown in Figure 3.10, converts the two signals into a binary state.

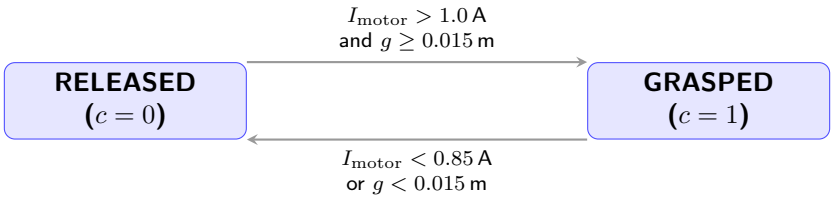


Figure 3.10. Hysteresis state machine for motor-current grasp detection.

The flag is raised when the current exceeds 1.0 A and is lowered below 0.85 A. Giving this 0.15 A gap prevents oscillations around the threshold. An aperture check overrides both: if the fingers are closer than 1.5 cm they are empty, whatever the current, which rules out the false positive produced by closing the gripper on nothing. The two current thresholds were determined empirically, through tests on the real robot.

Once the flag is raised, the preprocessor replaces the visual object pose in the observation vector with the end-effector pose obtained from the kinematic chain. The reasoning is that once the object is held, its pose is no longer an independent quantity: it is the gripper's pose, known exactly from proprioception, whereas the visual estimate at that moment is at its least reliable, since the object is largely hidden behind the fingers. When the flag drops, the preprocessor returns to the visual estimate.

3.3.4 Control Nodes

The three control nodes act one after the other at every control step, and together turn the latest measurements into a command for the robot.

Preprocessor. At start-up the node waits for a first reading of every controlled joint and a first estimate of the object pose, so that the policy never acts on an incomplete observation. Once these have arrived, it publishes an observation every 20 ms, whether or not new readings have come in since the previous one: each quantity simply takes the most recent value available. The observation follows the conventions of Section 3.1.4: the joint positions are expressed as displacements from their default values, and the yaw as the pair $[\cos\theta, \sin\theta]$. The position of the end-effector is obtained from the kinematic chain of the robot. The two flags of the observation are also handled here. The contact signal is the grasp flag of Section 3.3.3, and the reached flag is derived from it: it is raised when the object is held within 5 cm of the goal coordinates, and it is cleared as soon as the object is released.

Inferer. At every step the node evaluates the actor network on the last observation it has received and publishes the resulting action.

Postprocessor. Each component of the action is multiplied by the scale of the corresponding joint, which gives a displacement. During deployment it was found that these scales can also be modulated to produce satisfactory motions at different velocities by increasing or reducing them. The displacement is then added to the

current measured position of the joint, and the resulting absolute target is sent to the arm and gripper controllers as a trajectory point to be reached in 20 ms, that is one control step. The action is also sent back to the preprocessor, which uses it as the previous action in the next observation.

3.3.5 Execution Frequencies

Table 3.4 lists the rate and trigger of each topic in the loop. The control path is driven by wall timers at 50 Hz, matching the training decimation, while perception and grasp detection run at their own sensor-determined rates and are consumed asynchronously.

Table 3.4. Topic rates, sources and triggering mechanisms in the deployment architecture.

Topic	Rate	Source node	Trigger
`/preprocessor/obs`	50 Hz	`pal_policy_preprocessor`	Wall timer (20 ms)
`/inferer/output`	50 Hz	`pal_policy_inferer`	Wall timer (20 ms)
`/arm_right_controller/joint_trajectory`	50 Hz	`pal_policy_postprocessor`	Callback on `/inferer/output`
`/gripper_right_controller/joint_trajectory`	50 Hz	`pal_policy_postprocessor`	Callback on `/inferer/output`
`/predicted_object_pose`	15–30 Hz	`yolo_depth_pose_node`	Camera frame callback
`/grasped_current`	~ 87 Hz	`grasped_current_publisher`	Diagnostics callback
`/joint_states`	50–100 Hz	Hardware / Gazebo	Joint state controller

The time required by the detector was measured to verify that perception can keep up with the camera. On the GPU, at the deployment resolution of 240×320 pixels, a forward pass takes 16.05 ms on average. Since the camera delivers a frame every 33–67 ms, this leaves a wide margin for the remaining steps of the pipeline before the next frame arrives.

The camera stream is nevertheless slower than the control loop: a new object pose arrives every 33–67 ms, while an observation is assembled every 20 ms. This is not a problem. The preprocessor does not wait for the perception output, but reuses the most recent estimate until a newer one arrives, so that the same object pose can appear in up to four consecutive observations. Since the object stands still on the table during the approach, an estimate that is a few tens of milliseconds old is still valid, and once the object is grasped its pose is no longer taken from the camera but from the end-effector, as described in Section 3.3.3.

Chapter 4

Experimental Results

To validate the proposed approach, an extensive evaluation study was conducted. This chapter reports all evaluation phases, starting with an analysis of the results arising from the training plots. This is followed by extensive tests in **mjlab** to verify that the core manipulation goals are successfully achieved in simulation. Next, the perception pipeline and sim-to-sim transferability are validated in Gazebo by assessing pose tracking capabilities during the approach phase against exact ground-truth values. Finally, having established the validity of the overall approach, the full pipeline is deployed on the real hardware, concluding the chapter with an analysis of both qualitative and quantitative physical results.

4.1 Policy Training Metrics and Convergence Analysis

This section analyzes the policy's optimization dynamics and metric convergence over 15,000 training iterations within the GPU-accelerated **mjlab** (MuJoCo) simulation environment.

4.1.1 Task Reward Density Progression

By examining the per-step reward density components ($R_{\text{step}} = R_{\text{episode}}/T$) in Figure 4.1, we can observe how the chosen reward shaping strategy successfully establishes a physical curriculum. Through pure exploration, the agent learns to chronologically chain the required sub-skills: sequentially moving the end-effector closer to the object (*Reaching Object*), securing and elevating it (*Lifting Object*), transporting it towards the spatial goal (*Goal Tracking*), and finally letting it go (*Post-Reached Release*).

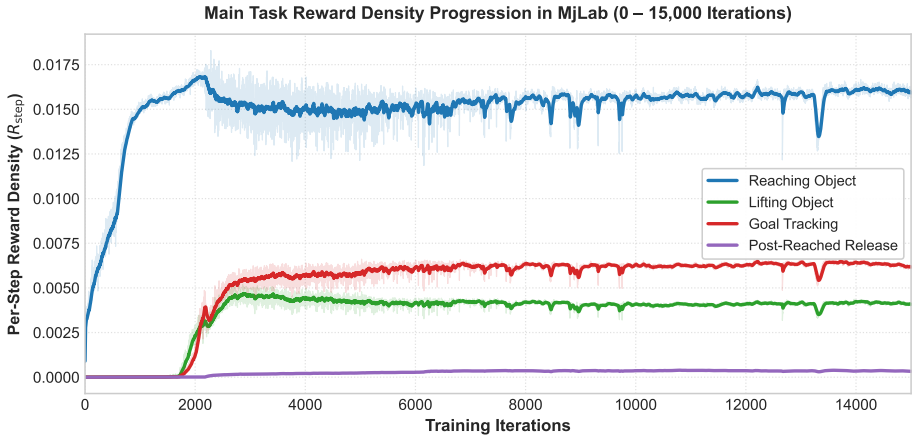


Figure 4.1. Progression of primary task rewards expressed as per-step reward density ($R_{\text{step}} = R_{\text{episode}}/T$) across 15,000 training iterations in **mjlab**.

As illustrated in Figure 4.1, the learning process naturally transitions through four sequential phases:

1. **Approach Phase (Iter** 0–2,000**)**: The policy rapidly optimizes *Reaching Object*, driving its density to a high peak as spatial alignment with the target cube is established.

2. **Grasp & Lift Phase (Iter** 1,800–2,500**)**: Discovery of dual-finger contact triggers a steep concurrent rise in *Lifting Object*, marking the onset of object elevation.

3. **Goal Transport Phase (Iter** 2,000–3,500**)**: As soon as the robot learns to lift the object, the *Goal Tracking* reward spikes sharply, signaling active spatial transport toward the destination.

4. **Release Phase (Iter** 2,500–15,000**)**: The policy learns to execute the *Post-Reached Release*, settling all primary task components into a steady-state equilibrium for long-horizon completion.

Furthermore, analyzing the relative magnitudes of these steady-state plateaus provides direct insight into the temporal nature and weight distribution of the reward shaping scheme. The *Reaching Object* reward maintains the highest overall density because it begins accumulating from the very first step of every episode. Conversely, while *Lifting Object* initially surges higher than transport during phase discovery, it eventually settles into a lower plateau (≈ 0.004) due to its minor proportional weight contribution. As soon as lifting is mastered, *Goal Tracking* asserts dominance, stabilizing at a higher magnitude (≈ 0.006) as it dictates the core transport behavior. Finally, the *Post-Reached Release* density appears deceptively small in the plot not because of task failure, but because it is exclusively accumulated during the final instants of an episode once the spatial goal has been reached.

4.1.2 Total Reward Density and PPO Loss Analysis

To rigorously evaluate the optimization dynamics and expose the inherent trade-offs in the actor-critic architecture, Figure 4.2 displays the per-step total reward density, the Actor surrogate loss, and the Critic value loss across 15,000 training iterations.

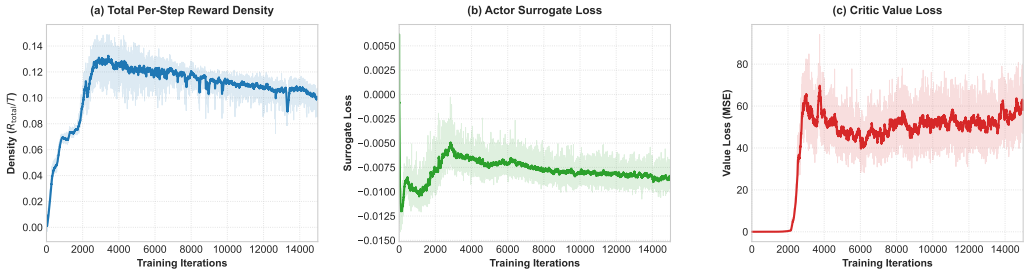


Figure 4.2. Training metrics across 15,000 iterations in **mjlab**: (a) per-step total reward density (R_{total}/T), (b) Actor surrogate loss, and (c) Critic value loss.

While the policy successfully masters the task, analyzing these three metrics provides valuable insights into the underlying PPO optimization dynamics:

- **Total Per-Step Reward Density** (Figure 4.2a): The global performance metric (R_{total}/T) ascends steeply to a peak near 0.13 (Iter 3,000), before stabilizing into a steady plateau at ≈ 0.10. This behavior reflects a natural property of dense reward shaping combined with early episode termination. Because the environment grants dense per-step rewards for reaching and lifting, an agent that rapidly completes the task forfeits additional steps of accumulable dense rewards. This natural trade-off between accumulating per-step rewards and reaching early task completion accounts for the slight density adjustment as the agent optimizes for time efficiency.

- **Actor Surrogate Loss** (Figure 4.2b): The surrogate loss settles into a narrow, small negative band (≈ -0.008), which indicates correct PPO behavior. This confirms that the clipping mechanism ($1 \pm \epsilon$, with $\epsilon = 0.2$) and KL adaptive learning rate function as intended, keeping policy updates strictly bounded to ensure a stable learning.

- **Critic Value Loss** (Figure 4.2c): The value loss stabilizes at a non-zero plateau ($\approx$ 55–60), which is expected and correct under domain randomization. Because initial object poses and goal targets are heavily randomized across episodes, different trajectories yield inherently different total returns, creating an inherent baseline variance that the Critic cannot reduce to zero.

4.2 Policy Performance Evaluation in mjlab

The trained policy capabilities were initially evaluated on **mjlab**, its own training environment, by running 1,000 trials under every different condition to assess task completion. Based on the adopted reward shaping strategy, the goal was to achieve

a policy capable of completing the full manipulation task while maintaining correct angular alignment with respect to the object's orientation.

The chosen metrics for this evaluation are:

- **Task Success Rate**: The percentage of episodes where the target object is successfully transported and released at the goal location without triggering penalty termination conditions.
- **Fingertip Alignment Error**: The absolute angular deviation between the gripper's approach yaw and the object's principal yaw at the moment of initial contact.

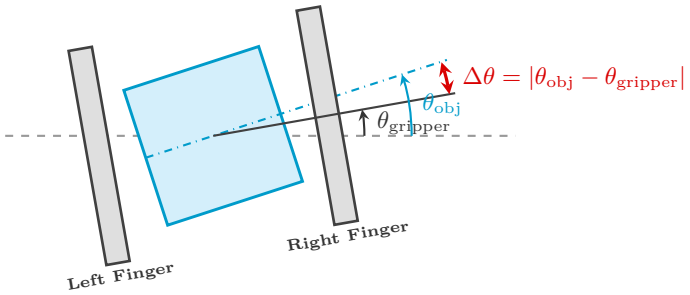


Figure 4.3. Schematic representation of the fingertip yaw alignment error ($\Delta\theta = |\theta_{\text{obj}} - \theta_{\text{gripper}}|$). The object's yaw angle θ_{obj} is measured between the horizontal reference axis and the cube's bottom edge, while θ_{gripper} denotes the orientation of the surrounding parallel fingertips.

4.2.1 Robustness and Generalization Analysis

In order to verify the robustness of the policy, 1,000 trial runs were conducted each time across distinct object geometries and observation noise regimes. The test conditions included:

- **Cuboids (Training Geometry)**: Randomly sized (width/depth: 2.0 cm to 5.0 cm; height: 4.0 cm to 8.0 cm) evaluated in the absence of noise.
- **Cuboids with** 1× **Noise**: Evaluated while injecting observation noise values matching those used during training.
- **Cuboids with** 2× **Noise**: Evaluated under double the training noise to stress-test the policy.
- **Cylinders (Unseen Geometry)**: Randomly sized (diameter: 2.5 cm to 5.0 cm; height: 4.0 cm to 7.0 cm) evaluated in the absence of noise. Cylinders were completely unseen during training, but due to the decoupled nature of the vision-control approach, the policy should theoretically be able to grasp any physically graspable object once its 3D coordinates are known.
- **Cylinders with** 1× **Noise**: Evaluated under standard training noise.
- **Cylinders with** 2× **Noise**: Evaluated under double the training noise.

Table 4.1 summarizes the quantitative metrics obtained across all tested configurations.

Table 4.1. Policy evaluation results in **mjlab** across 1,000 independent trials for Cuboid and Cylinder geometries under varying noise regimes.

Target Geometry	Observation Noise	Success Rate	Fingertip Alignment Error
Cuboids (Seen)	Nominal (0×)	**98.60%**	**3.87°**
	1× Training	**97.40%**	**4.20°**
	2× Double	**94.50%**	**4.83°**
Cylinders (Unseen)	Nominal (0×)	**97.30%**	**3.97°**
	1× Training	**95.40%**	**4.18°**
	2× Double	**84.10%**	**4.63°**

4.2.2 Evaluation Conclusions

The data unequivocally validates the efficacy of the manipulation policy. When operating under nominal and standard 1× training noise, the policy demonstrates exceptional robustness on the training geometry (Cuboids), maintaining success rates of 98.60% and 97.40%, respectively, alongside highly precise fingertip alignments.

More importantly, the results firmly establish the policy's capacity for **zero-shot generalization** to novel, unseen geometries (Cylinders). Because the Reinforcement Learning policy acts purely upon the abstracted 3D spatial coordinates and yaw orientation of the target, it successfully adapts to the curved geometric boundaries of cylinders without any retraining. Although rotationally symmetric objects such as cylinders do not physically possess a unique principal yaw orientation, computing the fingertip alignment error was nevertheless possible because the simulation environment continuously maintains and assigns an internal reference frame and yaw coordinate to the object body. The policy retains a 97.30% success rate without noise and a 95.40% success rate under standard training noise, accompanied by mean fingertip alignment errors ($\approx 4.0°$) that perfectly mirror those achieved on cuboids. Even if the cylinders don't present an orientation attribute, we were able to compute the fingertip alignment because the simulator always assign a yaw value.

When subjected to severe 2× noise, the policy still maintains a highly robust 94.50% success rate on flat-faced cuboids. However, a steeper degradation is observed on cylinders, dropping to 84.10%. This drop is physically expected: extreme coordinate corruption on curved, non-planar surfaces leaves a significantly narrower margin for error for parallel-jaw grippers than the flat, accommodating faces of a cuboid. Nevertheless, maintaining an 84.10% success rate on an entirely unseen geometry under extreme, beyond-training sensory noise serves as a strong empirical confirmation of the policy's adaptability and overall robustness.

4.3 Evaluation Through Gazebo Implementation

Deploying the pipeline in Gazebo prior to real hardware testing provides a critical intermediate validation step: the control structure used is nearly identical to the one run on the physical platform, the physics modeling is substantially more accurate

than in **mjlab**, and the resulting motions are far more representative of what the real robot will produce. This makes it possible to detect and correct potential sim-to-sim gaps before committing to hardware trials. Among these intermediate tests, the perception pipeline was evaluated by comparing predicted pose coordinates against exact ground-truth values during the approach phase, across nine object placements on the table with variable orientations.

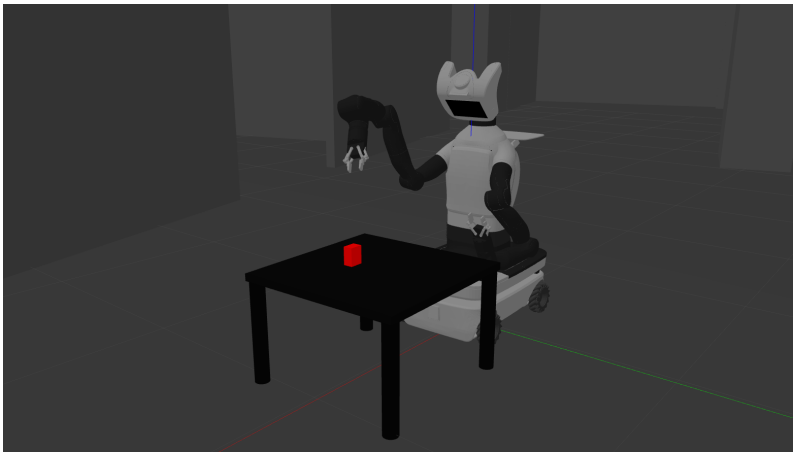


Figure 4.4. The TIAGo Pro model in the Gazebo simulation environment, facing a table with a target cuboid object to be grasped and transported.

4.3.1 3D Pose Estimation Accuracy vs. Ground-Truth

The spatial accuracy of the perception pipeline was systematically evaluated across nine target object configurations arranged in a 3×3 regular grid on the robot's workspace table, covering three radial distances (*close*, *middle*, and *far*) and three lateral positions (*left*, *center*, and *right*). Figure 4.5 shows the RViz2 visualization during one of these evaluation runs: the left panel displays the head camera feed with the estimated 3D position, yaw angle, YOLO detection confidence, and a yellow bounding square overlaid on the detected object; the right panel shows the corresponding robot model in the Gazebo scene along with a green dot representing the object's predicted coordinates and a red dot representing the goal coordinates.

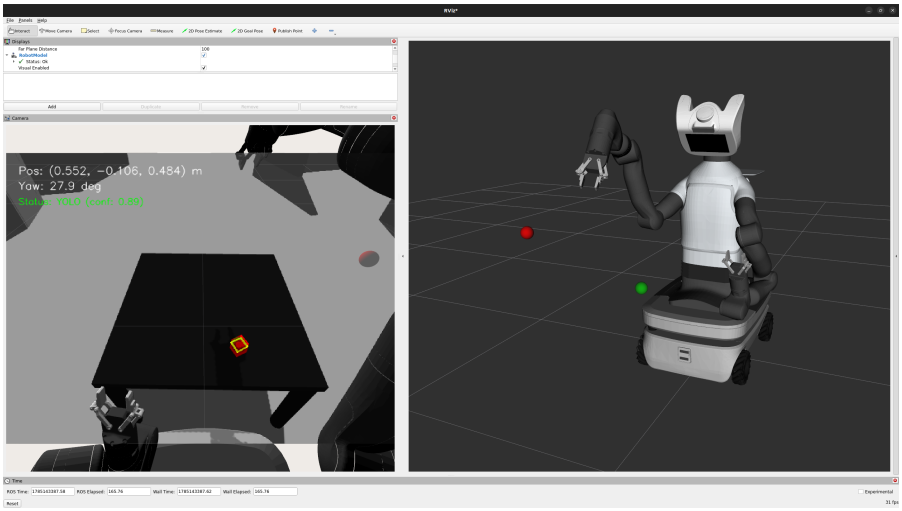


Figure 4.5. RViz2 visualization during Gazebo evaluation. Left: head camera feed with the estimated 3D position, yaw angle, YOLO detection confidence, and yellow bounding square overlaid on the detected object. Right: 3D robot model with a green marker indicating the predicted object position and a red marker indicating the goal coordinate.

For each configuration, two metrics were computed per coordinate component: the **Root Mean Square Error (RMSE)**, measuring overall prediction accuracy, and the **standard deviation**, used as an indicator of temporal stability: high values would suggest jittery, oscillating predictions that could destabilize the closed control loop. Table 4.2 reports both metrics across all components for the full nine-point evaluation grid.

Table 4.2. Root Mean Square Error (RMSE) and standard deviation of target pose predictions relative to Gazebo ground-truth across the nine-point evaluation grid.

Coordinate Component	Mean RMSE	Standard Deviation
X position (cm)	0.281	0.298
Y position (cm)	0.287	0.108
Z position (cm)	0.261	0.148
Yaw angle (°)	0.346	0.224

These results confirm the high spatial accuracy of the perception pipeline: an average positional RMSE below 3.0 mm across all three Cartesian axes and a mean orientation error of only 0.346° demonstrate that the vision node provides state estimates well within the tolerance margins required by the RL policy. To maintain temporal stability and mitigate prediction jitter, Exponential Moving Average (EMA) filters were applied to the raw pose outputs. The smoothing strength is governed by the filter parameter α: lower values of α yield stronger noise attenuation at the cost of increased latency, whereas higher values preserve responsiveness at the cost of residual jitter. The optimal α setting was determined empirically during these Gazebo trials, ensuring consistent pose tracking. Overall, these evaluations validate that the perception pipeline is both accurate and stable enough for physical hardware deployment, clearing the way for real-robot experiments.

4.4 Evaluation on the TIAGo Pro Platform

The final evaluation stage of this work validates the zero-shot deployment of the trained pick-and-place policy directly on the physical TIAGo Pro platform. This section details the physical experimental setup, the suite of test objects selected to evaluate generalization, a qualitative evaluation of task execution and sim-to-real operational challenges, and a quantitative analysis of closed-loop tracking precision, joint trajectory and effort telemetry.

4.4.1 Experimental Setup



(a) Physical experimental setup. The TIAGo Pro robot faces a table with a target object; a storage chest is placed at a fixed, pre-specified lateral position as the release target.



(b) Objects used in evaluation trials, spanning different sizes, surface textures, and colours to probe perception and manipulation generalization.

Figure 4.6. Real-robot evaluation environment (left) and the set of test objects (right).

The experiment replicates the simulation task structure in the physical world (Figure 4.6). The robot is placed in front of a table at a fixed base position. A target object is placed on the tabletop at a random location within the robot's reachable workspace. The task is defined as follows:

1. The perception pipeline (Section 3.2) detects the object from the head RGB-D camera stream, estimates its 3D pose, and tracks it in real time.

2. The policy receives the estimated object position and a fixed 3D *goal coordinate* corresponding to a point above a storage chest placed at a pre-defined location to the robot's right.

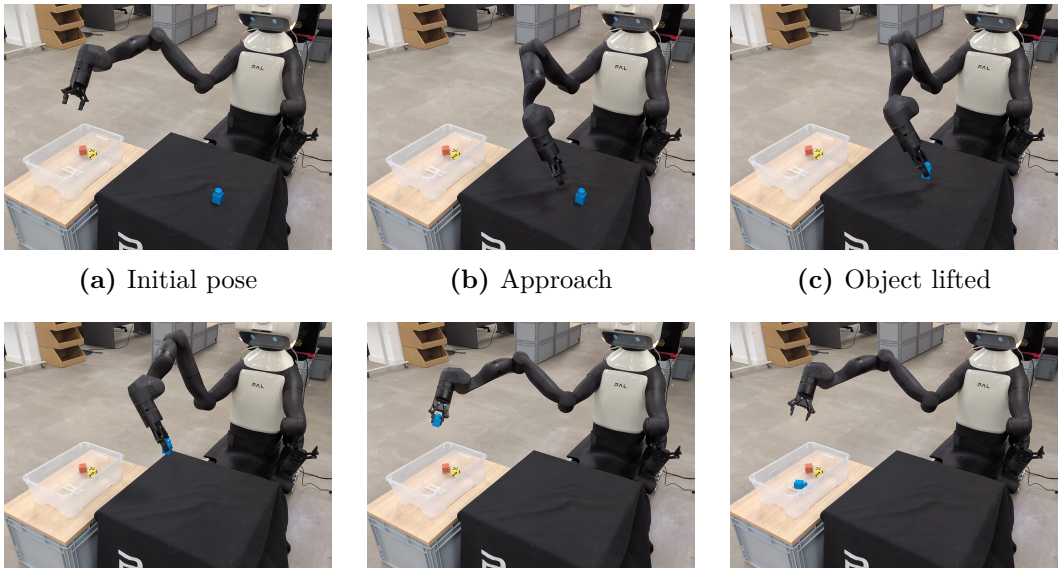
3. The robot must approach, grasp, and transport the object to the chest, then open the gripper to release it inside.

The suite of evaluation objects (Figure 4.6b) was deliberately curated to span a diverse range of visual and physical characteristics, including varying colors, surface textures, geometries, and scales. Crucially, the decoupled perception-control

architecture, combined with reward shaping terms that incentivize maximum fingertip aperture during approach, enables broad object-agnostic generalization. Consequently, as long as an object detector such as YOLO identifies the target, it is seamlessly processed by the perception pipeline to feed the estimated state into the RL policy, enabling the robot to manipulate any newly introduced object fitting within the gripper's physical limits.

4.4.2 Qualitative Evaluation and Operational Observations

The results shown during simulation evaluations reflect congruently in physical hardware trials. Through zero-shot sim-to-real transfer, the trained policy enabled the TIAGo Pro platform to reliably execute the full pick-and-place task in the real world without hardware fine-tuning or adaptation. Crucially, experimental testing confirmed the repeatability of the learned behavior: the robot demonstrated the ability not only to perform a single pick-and-place operation but also to execute continuous, repeated trials by returning to pick up a new item from the table immediately after releasing the previously held object. Figure 4.7 illustrates a sequence of keyframes capturing a complete execution cycle.



(a) Initial pose **(b)** Approach **(c)** Object lifted

(d) Reaching release point **(e)** Ready to release **(f)** Object released

Figure 4.7. Sequential video keyframes illustrating the physical TIAGo Pro executing the real-world pick-and-place task: (a) initial pose following the release of a previous cube, (b) vision-guided approach towards the target cube, (c) object grasp and vertical elevation, (d) reaching the release coordinates above the chest, (e) pre-release positioning, and (f) object released inside the storage chest.

4.4.3 Sim-to-Real Discrepancies and Operational Limitations

Despite the high task success rate, the primary sim-to-real discrepancy observed during physical hardware trials pertained to the task execution velocity. With the same action scale values, the policy executed motion at a significantly higher velocity

in simulation whereas physical deployment on the TIAGo Pro platform resulted in visibly slower motion. The principal cause of this discrepancy lies in the physical dynamic parameters defined within the MuJoCo MJCF model used for reinforcement learning. Because RL applications within PAL Robotics were historically tailored primarily for humanoid platforms, the TIAGo Pro model parameters (such as joint damping, armature inertia, and actuator friction) had not yet undergone rigorous hardware-level system identification. Further empirical investigation revealed that the real-world execution speed could be arbitrarily adjusted by tuning the policy's action scale parameter; however, increasing the action scale introduced a slightly more jittery and oscillatory joint motion, though remaining strictly within acceptable operational limits.

Another practical limitation identified during experimental trials concerned the orientation estimation module within the perception pipeline, particularly when tracking smaller target objects. During live task execution, the raw predicted yaw angle frequently exhibited substantial temporal oscillations, occasionally leading to misaligned approach vectors. This instability occurred because the target objects appeared very small in the camera view (occupying only a few pixels), making the geometric orientation estimation algorithm highly sensitive to minor image noise. To resolve this, alternative learning-based solutions were investigated, including training a Convolutional Neural Network (CNN) for orientation regression using synthetic Gazebo images rendered under extensive visual domain randomization, but these models failed to achieve reliable zero-shot real-world generalization. Consequently, the yaw lock described in Section 3.3.2 was adopted. Although it prevents the orientation estimate from adapting if the object is displaced during execution, this limitation proved minor in practice. Crucially, the 3D spatial position estimation remained continuously active and unaffected, while the small physical size of the target objects allowed the parallel-jaw gripper to reliably enclose and capture them even in the presence of residual orientation uncertainties.

4.4.4 Quantitative Execution Analysis

During physical task execution, comprehensive telemetry was recorded to evaluate system performance, joint trajectory smoothness, and actuator torque requirements relative to hardware limits. The time-series profiles presented in this section capture the telemetry logged during a representative pick-and-place trial.

End-Effector Distance to Predicted Object and Goal Coordinates

To assess tracking behaviour throughout the task execution, we log the positions of the end-effector (EE) and the perception-estimated object centroid along the three Cartesian axes (X, Y, Z) independently. Red dashed lines mark the fixed target goal coordinates ($X = 0.71\,\text{m}$, $Y = -0.65\,\text{m}$, $Z = 0.68\,\text{m}$). The orange background highlights the *grasped* phase (contact detected), while purple background indicates the combined *grasped + reached* phase, which activates when the 3D Euclidean distance between the end-effector and the goal drops below the 5 cm threshold ($< 0.05\,\text{m}$).

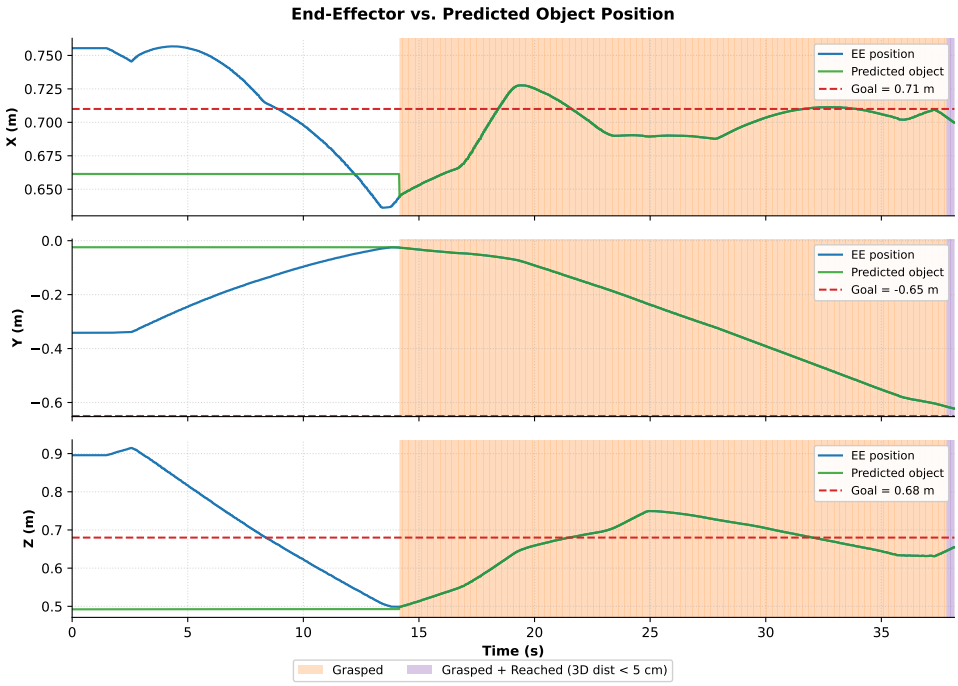


Figure 4.8. End-effector position (blue) and perception-estimated object centroid (green) along X (top), Y (middle), and Z (bottom) axes during a representative deployment trial on the physical TIAGo Pro.

As demonstrated by these trajectories, the end-effector smoothly converges toward the target object during the vision-guided approach phase, secures the grasp, and steadily transports the held object toward the goal coordinates until the reaching threshold ($< 5.0\,\text{cm}$) is satisfied for object release.

Joint Positions During Task Execution

Joint position trajectories provide a direct measure of motion smoothness and confirm that the policy avoids abrupt joint reversals or high-frequency oscillations that would indicate instability at the sim-to-real boundary. Figure 4.9 presents eight vertically stacked subplots displaying the trajectory of each individual right-arm joint angle (q_1–q_7, in radians) alongside the gripper finger displacement (g, labelled g_{pos} in the plot, in meters) over the course of the task execution.

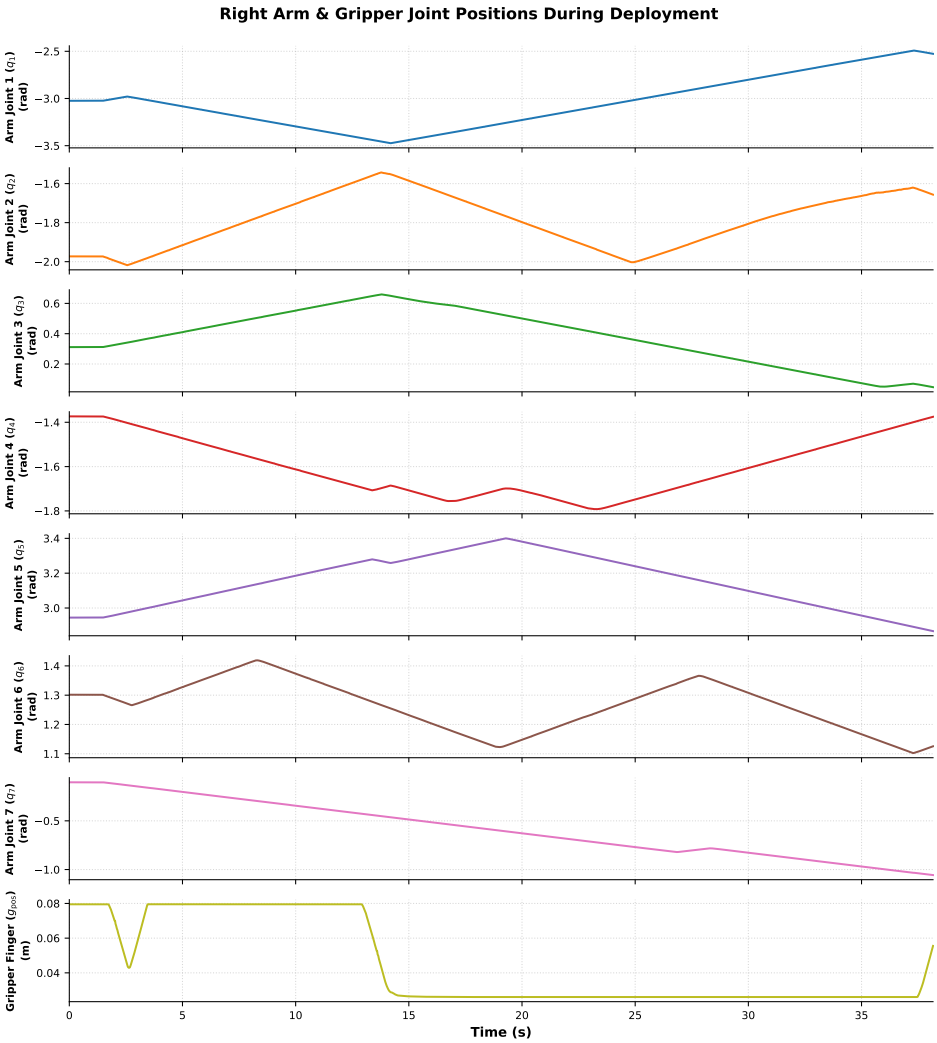


Figure 4.9. Right-arm joint positions (q_1–q_7, rad) and parallel gripper finger displacement (g_{pos}, m) plotted across eight separate stacked subplots during a representative physical deployment trial on TIAGo Pro.

As evidenced by these position trajectories, all seven arm joints exhibit smooth, continuous motion avoiding high-frequency oscillations, while the gripper finger displacement transitions cleanly from maximum opening (7.95 cm) to firm object closure (2.61 cm) upon contact.

Joint Effort Requirements During Task Execution

Joint effort (torque) profiles are analysed to verify that the policy operates safely within the actuator limits of the physical hardware. Figure 4.10 presents seven vertically stacked subplots tracking the torque requirements (τ_1–τ_7, in N·m) of each arm joint. Red dashed lines mark the defined actuator torque limits (±43.0 N·m for joints 1–2, and ±26.0 N·m for joints 3–7). Gripper effort telemetry and specified force limits are omitted as they are not exposed by the physical hardware controller interface.

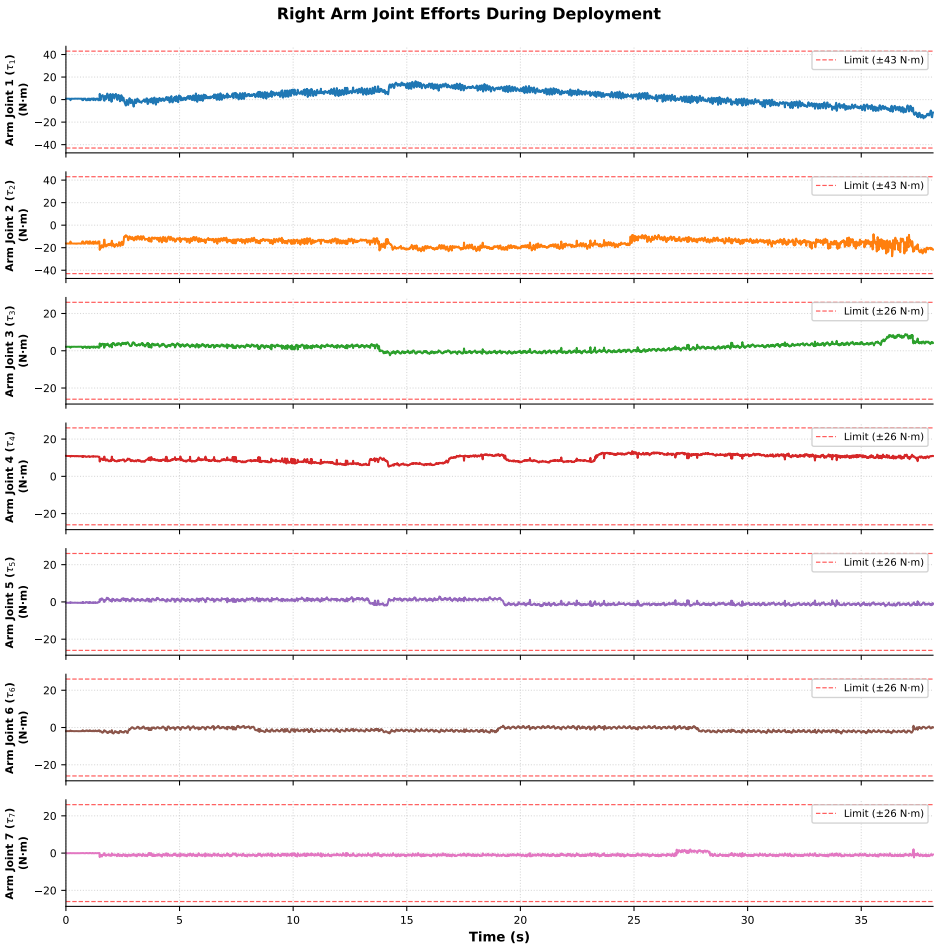


Figure 4.10. Right-arm joint efforts (τ_1–τ_7, N·m) across seven separate stacked subplots during real-robot deployment. Horizontal red dashed lines mark the rated continuous-torque limits for each joint.

All measured torques remain strictly within their safety envelopes, confirming that the policy executes smooth, energy-efficient control without overloading the actuators.

Chapter 5

Conclusions and Future Work

5.1 Conclusions

This thesis presented a method to complete a pick-and-place task with the TIAGo Pro platform in its out-of-the-box configuration. The robot is controlled by a reinforcement learning policy, which receives the proprioception of the robot together with an estimate of the pose of the object. This estimate is produced by a perception pipeline that combines an AI-based detector with classical image processing techniques.

As shown in Chapter 4, the method was validated in progressive steps. First, the policy was evaluated in the **mjlab** simulation environment under realistic observation noise. Then the perception pipeline, which turns the image stream of a head-mounted RGB-D camera into predictions of the object pose, proved effective in the Gazebo simulation. The two parts were then tested together in Gazebo through a ROS 2 control architecture, a step that proved essential for a first assessment of the sim-to-real transferability.

Finally, the tests on the real hardware demonstrated zero-shot transfer, and validated the ability of the complete method to make the TIAGo Pro approach, grasp, transport and deposit a wide variety of objects in a chest.

5.2 Limitations

Although the task was executed successfully, some limitations of the presented approach emerged during the validation steps.

The first is not specific to the approach, but concerns an aspect of great importance in any training based on simulation: the accuracy of the model. Although sufficient for the purposes of this project, the dynamic parameters of the model used to simulate the TIAGo Pro in **mjlab** proved not to be fully accurate. The main consequence was a large discrepancy between the speed at which the task is executed in **mjlab** and the one observed both in the Gazebo simulator and on the real platform, with execution times being particularly long on the real hardware. The Gazebo model, which proved to be particularly faithful to the behaviour of the real robot, could not be used for training directly since **mjlab** is based on MuJoCo, whose MJCF format describes the dynamics of the joints with its own parameters:

some of them, such as the armature, have no counterpart in the URDF description used by Gazebo, and those that do, such as the joint friction, are handled differently by the two physics engines, so that the values cannot simply be copied from one model to the other. Future work on this platform within the **mjlab** framework will therefore probably require an update of the MJCF model through some model identification techniques.

Further limitations concern the perception approach, and emerged during the deployment on the real platform.

The first lies in the yaw estimation strategy, which proved considerably less reliable than the position estimation. The rectangle fitting method is prone to errors: if just a few outliers pass the filtering steps, the fitted rectangle may still give a valid position centroid but an unreasonable orientation. In addition, when the end-effector gets close to the object it partially occludes it, which degrades the orientation estimate. For this reason the problem was simplified by locking the yaw estimate to the median of the values collected in the first three seconds of the task. This choice, however, makes the robot unresponsive to changes in the orientation of the object, such as those caused by a failed grasp attempt during the execution of the task. Given the main goal of the task and the variety of objects used in the tests, mostly of small size, this behaviour did not prove critical. As mentioned in Section 4.4.3, some methods based on neural networks were also tested to estimate the orientation, but with unsatisfactory results. The problem therefore remains open, and will have to be addressed in future work, where greater precision may matter more.

The second lies in the colour segmentation stage, which assumes the object to be of a single colour, as stated in Section 3.2.3. Since the reference colour is sampled from the centre of the bounding box, an object with several colours or with a pattern would be segmented only in part, and the pose estimated from the remaining pixels would be affected.

5.3 Future Work

5.3.1 Transfer to the Kangaroo Platform

The main reason behind this project was to establish a solid preliminary approach to the execution of manipulation tasks, which could later serve as a reference for similar work on the Kangaroo platform. The proposed approach proved effective, and no particular obstacle stands in the way of its transfer: the perception pipeline needs only an RGB-D camera, and the deployment architecture only a policy that maps a state vector to joint commands.

The Kangaroo is, however, a more demanding platform. Being a humanoid, it has a much larger number of joints to control and it has to keep its balance while it manipulates. Its legs, moreover, are not simple serial chains, since they are driven by linear actuators through closed kinematic chains. All of this makes training considerably more expensive than on the TIAGo Pro. For this reason the decoupled approach appears to be a sensible choice for a first manipulation strategy based on reinforcement learning on that platform: adding camera rendering to the **mjlab**

simulation would increase the computational cost drastically, whereas a policy that receives only the object's pose keeps the training as light as it was in this work.

5.3.2 Further Work on the TIAGo Pro

Without altering the core strategy, the work presented here could be extended in two ways on the TIAGo Pro. The first is to add an autonomous navigation subtask, so that the robot reaches the table by itself before starting the manipulation. The second is to actuate the joints that were kept fixed in this work, namely those of the head, of the torso and of the left arm, in addition to those of the right arm, which would extend the workspace of the robot.

It would also be interesting to explore other solutions based on reinforcement learning for the same task, starting from the camera setup. A camera mounted on the wrist observes the object from a few centimetres away, and therefore provides a much larger and unoccluded view during the final approach, which is exactly where the head camera loses sight of the object. The two cameras could also be combined: the head camera would give information on the whole workspace, while the wrist camera would allow a much more precise viewpoint.

A wrist camera would also make it possible to explore vision-based solutions, in which the camera is used to extract embedded features that are fed directly to the policy. Since the networks tested in this work to regress the orientation explicitly did not give satisfactory results, this more abstract path may be the answer to the orientation problem: instead of an explicit angle, the policy would receive a compact vector of features describing the object, and learn by itself how to use it to align the gripper. The price is that visual information enters the policy again, so that both the gap between simulated and real images, which the present design avoids, and the cost of rendering during training would have to be dealt with.

Bibliography

[1] Albardaner, J., San Miguel, A., García, N., and Dalmau-Moreno, M. Sim-to-real gap in RL: Use case with TIAGo and Isaac Sim/Gym. *arXiv preprint arXiv:2403.07091*, (2024).

[2] Andrychowicz, M., et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, **39** (2020), 3.

[3] Chi, C., Feng, S., Pan, Y., Feng, E., Xu, H., Cousineau, E., Burchfiel, B., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)* (2023).

[4] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pp. 1861–1870 (2018).

[5] Handa, A., et al. DeXtreme: Transfer of agile in-hand manipulation from simulation to reality. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5977–5984 (2023).

[6] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., and Hutter, M. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, **4** (2019), eaau5872.

[7] Jauhri, S., Peters, J., and Chalvatzaki, G. Robot learning of mobile manipulation with reachability behavior priors. *IEEE Robotics and Automation Letters*, **7** (2022), 8399.

[8] Jauhri, S., Prasad, V., and Chalvatzaki, G. Whole-body mobile manipulation using offline reinforcement learning on sub-optimal controllers. *arXiv preprint arXiv:2604.12509*, (2026).

[9] Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F. C., and Kalfaoglu, M. E. Ultralytics YOLO26: Unified real-time end-to-end vision models. *arXiv preprint arXiv:2606.03748*, (2026).

[10] Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, **101** (1998), 99.

[11] Kalashnikov, D., et al. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning (CoRL)*, pp. 651–673 (2018).

[12] Kroemer, O., Niekum, S., and Konidaris, G. A review of robot learning for manipulation: Challenges, representations, and algorithms. *Journal of Machine Learning Research*, **22** (2021), 1.

[13] Levine, S., Finn, C., Darrell, T., and Abbeel, P. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, **17** (2016), 1.

[14] Liu, P., Zhang, K., Tateo, D., Jauhri, S., Hu, Z., Peters, J., and Chalvatzaki, G. Safe reinforcement learning of dynamic high-dimensional robotic tasks: Navigation, manipulation, interaction. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1739–1745. IEEE (2023).

[15] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, **7** (2022), eabm6074.

[16] Mittal, M., et al. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, (2025).

[17] Mnih, V., et al. Human-level control through deep reinforcement learning. *Nature*, **518** (2015), 529.

[18] Müller, M., Dosovitskiy, A., Ghanem, B., and Koltun, V. Driving policy transfer via modularity and abstraction. In *Conference on Robot Learning (CoRL)*, pp. 1–15. PMLR (2018).

[19] Nair, S., Rajeswaran, A., Kumar, V., Finn, C., and Gupta, A. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning (CoRL)* (2022).

[20] Pagès, J., Marchionni, L., and Ferro, F. TIAGo: the modular robot that adapts to different research needs. In *International Workshop on Robot Modularity at IROS* (2016).

[21] PAL Robotics. TIAGo Pro technical specifications (datasheet). `https://pal-robotics.com/wp-content/uploads/2024/05/Datasheet-TIAGo-Pro.pdf` (2024). Accessed: 2026-09-18.

[22] Peng, X. B., Andrychowicz, M., Zaremba, W., and Abbeel, P. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803–3810 (2018).

[23] Peters, J. and Schaal, S. Reinforcement learning of motor skills with policy gradients. *Neural Networks*, **21** (2008), 682.

[24] Pinto, L., Andrychowicz, M., Welinder, P., Zaremba, W., and Abbeel, P. Asymmetric actor critic for image-based robot learning. In *Robotics: Science and Systems (RSS)* (2018).

[25] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788 (2016).

[26] Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)* (2016).

[27] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, (2017).

[28] Schwarke, C., Mittal, M., Rudin, N., Hoeller, D., and Hutter, M. RSL-RL: A learning library for robotics research. *arXiv preprint arXiv:2509.10771*, (2025).

[29] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edn. (2018).

[30] Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 12, pp. 1057–1063 (1999).

[31] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30. IEEE (2017).

[32] Todorov, E., Erez, T., and Tassa, Y. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE (2012).

[33] Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, **8** (1992), 229.

[34] Zakka, K., Liao, Q., Yi, B., Le Lay, L., Sreenath, K., and Abbeel, P. mjlab: A lightweight framework for GPU-accelerated robot learning. *arXiv preprint arXiv:2601.22074*, (2026).

[35] Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)* (2023).

[36] Zitkovich, B., et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)* (2023).